The Open Group Preliminary Standard

O-PAS™ Standard, Version 2.1

Part 6.6 – Information and Exchange Models: IEC 61131

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Copyright © 2021, The Open Group

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior permission of the copyright owners.

Any use of this publication for commercial purposes is subject to the terms of the Annual Commercial License relating to it. For further information, see www.opengroup.org/legal/licensing.

The Open Group Preliminary Standard

O-PAS™ Standard, Version 2.1: Part 6.6 – Information and Exchange Models: IEC 61131

Document Number: P210-6f

Published by The Open Group, May 2021.

Comments relating to the material contained in this document may be submitted to:

The Open Group, Apex Plaza, Forbury Road, Reading, Berkshire, RG1 1AX, United Kingdom

or by electronic mail to:

ogspecs@opengroup.org

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Contents

1 Introduction 1
- 1.1 Objective 1
- 1.2 Overview 1
- 1.3 Conformance 1
- 1.4 Normative References 1
- 1.5 Terminology 3
- 1.6 Future Directions (Informative) 3

2 Terms and Definitions 4
- 2.1 OPAF Glossary and Abbreviations 4
- 2.2 Terms Specific to Part 6.6 4

3 Information Model (Informative) 5
- 3.1 Abstract Information Model 5
- 3.2 IEC 61131-3 Constraints (Informative) 17

4 Import/Export Format 19
- 4.1 Import/Export Format Structure 19
- 4.2 Open Packaging Conventions 19
- 4.3 IEC 61131-10 AddData 19
  - 4.3.1 UAObjectTypeLink 19
  - 4.3.2 UAObjectVariableLink 20
  - 4.3.3 UAObjectVariableAttributeLink 21
- 4.4 OPC UA Nodeset file 22
- 4.5 AutomationML 23
  - 4.5.1 61131-XMLFileRelationship Interface Class 23
  - 4.5.2 61131-Engine Role Class 23
  - 4.5.3 61131-Application Role Class 23
  - 4.5.4 61131-FunctionBlockGroup Role Class 23
  - 4.5.5 System Unit Classes 23
  - 4.5.6 InstanceHierarchy 24
- 4.6 Import/Export Constraints 24
- 4.7 Import/Export Profiles, Facets, and Conformance Units 25
  - 4.7.1 Profiles 25
  - 4.7.2 Conformance Units 25

5 Download/Upload Exchange Format 26

6 OCF Information Model 27
- 6.1 O-PAS OPC UA Object Types 27
  - 6.1.1 OPAS61131ExecutionEngineType 28
  - 6.1.2 OPAS61131FunctionBlockGroupType 29
- 6.2 Type AddressSpace 29
  - 6.2.1 Rules for OPC UA Object Types for IEC 61131-3 Programs, Function Blocks, and Classes 29
- 6.3 Instance AddressSpace 30

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

6.4 Namespaces 30
6.4.1 Namespace Metadata 30
6.4.2 Handling of OPC UA Namespaces 31
6.5 Mapping of IEC 61131-3 Data Types to UA Variables and Data Types 33
6.5.1 Non O-PAS Struct Variables 34
6.5.2 O-PAS Struct Variables 34
6.6 OPC UA Conformance Units 35
6.6.1 Conformance Units 35

7 Function Blocks and IEC 61131 Extensions 37
7.1 Function Blocks Defined in O-PAS Part 6.4 37
7.2 O-PAS Connectivity Framework Blocks (OCFBs) 37
7.2.1 Overview 37
7.2.2 Function Block: OCFB_Trigger_IN 41
7.2.3 Function Block: OCFB_Trigger_OUT 42
7.2.4 Function Block: OCFB_IN_<Type> 43
7.2.5 Function Block: OCFB_OUT_<Type> 44
7.3 Units of Measure 45
7.3.1 Function: UOM_NS_ID() 45
7.4 IEC 61131-3 NAMESPACEs 46
7.5 Function Block Conformance Units and Profiles 46
7.5.1 Conformance Units 46
7.5.2 Profiles 47

A Data Types (Normative) 48
A.1 OPAS_BOOL_SIGNAL_TYPE 48
A.2 OPAS_LINT_SIGNAL_TYPE 48
A.3 OPAS_ULINT_SIGNAL_TYPE 49
A.4 OPAS_REAL_SIGNAL_TYPE 50
A.5 OPAS_LREAL_SIGNAL_TYPE 50
A.6 OPAS_SIGNAL_METADATA_TYPE 51
A.7 OPAS_SIGNAL_STATUS_TYPE 51
A.8 OPAS_RANGE_REAL_TYPE 52
A.9 OPAS_RANGE_LREAL_TYPE 52
A.10 OPAS_EU_INFO_TYPE 53
A.11 OPAS_SIGNAL_SEVERITY_TYPE 53
A.12 OPAS_SIGNAL_STATUS_SUBCODE_TYPE 54
A.13 OPAS_SIGNAL_INFO_TYPE 54
A.14 OPAS_SIGNAL_INFOBITS_TYPE 55
A.15 OPAS_SIGNAL_LIMITBITS_TYPE 55
A.16 OPAS_ACCESS_LEVEL_TYPE 56

B Helper Functions (Normative) 57
B.1 OPAS_SIGNAL_STATUS_IS_GOOD() 57
B.2 OPAS_SIGNAL_STATUS_IS_UNCERTAIN() 57
B.3 OPAS_SIGNAL_STATUS_IS_BAD() 58
B.4 OPAS_SIGNAL_STATUS_GET_SUBCODE() 58
B.5 OPAS_SIGNAL_STATUS_SET_SUBCODE() 59
B.6 OPAS_SIGNAL_STATUS_IS_DATAITEM() 59
B.7 OPAS_SIGNAL_STATUS_GET_LIMITS() 60
B.8 OPAS_SIGNAL_STATUS_SET_LIMITS() 60

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

C IEC 61131-10 AddData (Normative) 62
C.1 XML Schema Definition (xsd) 62
C.2 AddDataInfo 64
D O-PAS AutomationML Definitions (Normative) 65
E O-PAS Part 6.6 OPC UA Mappings (Normative) 66
E.1 Namespace and Identifiers for the O-PAS Part 6.6 Information Model 66
E.2 Profile URIs for the O-PAS Information Model 67
F Example of Linking Between IEC 61131-10 XML and OPC UA Nodeset2 XML (Informative) 68
F.1 Example IEC 61131-3 Function Block and IEC 61131-10 XML Description 69
G Abbreviations 73
G.1 Abbreviations Specific to Part 6.6 73

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Preface

The Open Group

The Open Group is a global consortium that enables the achievement of business objectives through technology standards. Our diverse membership of more than 800 organizations includes customers, systems and solutions suppliers, tools vendors, integrators, academics, and consultants across multiple industries.

The mission of The Open Group is to drive the creation of Boundaryless Information Flow™ achieved by:

- Working with customers to capture, understand, and address current and emerging requirements, establish policies, and share best practices
- Working with suppliers, consortia, and standards bodies to develop consensus and facilitate interoperability, to evolve and integrate specifications and open source technologies
- Offering a comprehensive set of services to enhance the operational efficiency of consortia
- Developing and operating the industry's premier certification service and encouraging procurement of certified products

Further information on The Open Group is available at www.opengroup.org.

The Open Group publishes a wide range of technical documentation, most of which is focused on development of Standards and Guides, but which also includes white papers, technical studies, certification and testing documentation, and business titles. Full details and a catalog are available at www.opengroup.org/library.

This Document

This document is Part 6.6 of the O-PAS™ Standard, Version 2.1, a Preliminary Standard of The Open Group. It has been developed and approved by The Open Group.

The O-PAS Standard consists of the following 12 parts (of the anticipated 13 parts):

- O-PAS Part 1 – Technical Architecture Overview (Informative)
- O-PAS Part 2 – Security
- O-PAS Part 3 – Profiles (Informative)
- O-PAS Part 4 – O-PAS Connectivity Framework (OCF)
- O-PAS Part 5 – System Management
- O-PAS Part 6.1 – Information and Exchange Models: Overview and Interfaces

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

- O-PAS Part 6.2 – Information and Exchange Models: Basic Configuration
- O-PAS Part 6.3 – Information and Exchange Models: Alarm and Events Configuration
- O-PAS Part 6.4 – Information and Exchange Models: Function Blocks
- O-PAS Part 6.5 – Information and Exchange Models: IEC 61499
- O-PAS Part 6.6 – Information and Exchange Models: IEC 61131 (this document)
- O-PAS Part 7 – Distributed Control Platform

The parts listed above are separate documents that can be updated and re-versioned as required as we move forward with the O-PAS Standard.

The O-PAS Standard, Version 2.1 is being published initially as a Preliminary Standard since it addresses an emerging area of technology; therefore, it may change before being published as a full standard of The Open Group. In such a case it will be made as upwards-compatible as possible with the corresponding Preliminary Standard, but complete upwards-compatibility is not guaranteed.

Once this document becomes a full standard, it will supersede the O-PAS Standard, Version 1.0 published in December 2019.

Conventions

A Glossary and Abbreviations reference is available. If a term is not defined in that document then the common English definition, as defined in Merriam-Webster's Collegiate Dictionary, applies.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Trademarks

ArchiMate, DirecNet, Making Standards Work, Open O logo, Open O and Check Certification logo, Platform 3.0, The Open Group, TOGAF, UNIX, UNIXWARE, and the Open Brand X logo are registered trademarks and Boundaryless Information Flow, Build with Integrity Buy with Confidence, Commercial Aviation Reference Architecture, Dependability Through Assuredness, Digital Practitioner Body of Knowledge, DPBoK, EMMM, FACE, the FACE logo, FHIM Profile Builder, the FHIM logo, FPB, Future Airborne Capability Environment, IT4IT, the IT4IT logo, O-AA, O-DEF, O-HERA, O-PAS, Open Agile Architecture, Open FAIR, Open Footprint, Open Process Automation, Open Subsurface Data Universe, Open Trusted Technology Provider, OSDU, Sensor Integration Simplified, SOSA, and the SOSA logo are trademarks of The Open Group.

Microsoft and Visual Studio are registered trademarks of Microsoft Corporation in the United States and/or other countries.

PA-DIM is a trademark of FieldComm Group, Inc.

All other brands, company, and product names are used for identification purposes only and may be trademarks that are the sole property of their respective owners.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Acknowledgements

The Open Group gratefully acknowledges and thanks the OPC Foundation for the template used to develop an OPC UA Companion Specification that is included in this O-PAS Standard. Visit opcfoundation.org.

The Open Group gratefully acknowledges the contribution of the following people in the development of this document:

Lead Contributors

- Jeff Harding – ABB Automation GmbH
- Lars Ott – CODESYS Group (Subcommittee Lead)

Other Contributors

- Dennis Brandl – BR&L Consulting, Inc.
- David Fort – Rockwell Automation
- Jim Luth – Schneider Electric
- Alexander Johnston – Schneider Electric
- Vien Nguyen – Yokogawa Electric Corporation
- Carl Schumaker – Rockwell Automation
- Eelco van der Wal – PLCopen

The Open Group gratefully acknowledges the contribution and leadership of the Co-Chairs of the Open Process Automation™ Forum and Working Groups.

Forum Co-Chairs

- Don Bartusiak – Collaborative Systems Integration, LLC
- Trevor Cusworth – Schneider Electric

Business Working Group Co-Chair

- Luis Duran – ABB Automation GmbH
- Jacco Opmeer – Shell

Enterprise Architecture Work Grouping Co-Chairs

- Dave Emerson – Yokogawa Electric Corporation
- Harirajan Padmanabhan – hpad, Inc.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Technology Working Group Co-Chairs

- David Fort – Rockwell Automation
- Kirk Smith – Intel Corporation

Certification Working Group Co-Chairs

- Ed Agis – Intel Corporation
- Ron Breault – Wind River Systems, Inc.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Referenced Documents

The following documents are referenced in Part 6.6 of the O-PAS Standard.

The documents are referred to in the text in such a way that some or all of their content constitutes requirements of this part. For dated references, only the version cited applies. For undated references, the latest version of the referenced document (including any amendments) applies.

(Please note that the links below are good at the time of writing but cannot be guaranteed for the future.)

Normative References

Normative references for Part 6.6 of the O-PAS Standard are defined in Section 1.4.

Informative References

- IEC 61499-1:2012: Function Blocks – Part 1: Architecture; refer to: https://webstore.iec.ch/publication/5506

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

1 Introduction

1.1 Objective

The objective of Part 6.6 of the O-PAS Standard is to define the import/export information model used by IEC 61131-3 application development tools, and the OPC UA nodes that an IEC 61131-3 O-PAS application must provide in its server.

1.2 Overview

This document outlines the integration of Distributed Control Nodes (DCNs) that support IEC 61131-3 into O-PAS networks.

1.3 Conformance

Part 6.6 of the O-PAS Standard is normative, and conformance requirements (marked with "CRQ" identifiers) are defined in Chapters 4, 6, and 7 and Appendices A, B, and C.

1.4 Normative References

The following standards contain provisions which, through references in this document, constitute provisions of Part 6.6 of the O-PAS Standard. At the time of publication, the editions indicated were valid. All standards are subject to revision, and parties to agreements based on this document are encouraged to investigate the possibility of applying the most recent editions of the standards listed below.

- IEC 61131-1:2003: Programmable Controllers – Part 1: General Information; refer to: https://webstore.iec.ch/publication/4550
- IEC 61131-3:2013: Programmable Controllers – Part 3: Programming Languages; refer to: https://webstore.iec.ch/publication/4552
- IEC 61131-5:2000: Programmable Controllers – Part 5: Communications; refer to: https://webstore.iec.ch/publication/4554
- IEC 61131-10:2019: Programmable Controllers – Part 10: PLCopen XML Exchange Format; refer to: https://webstore.iec.ch/publication/33034
- IEC 62714-1:2018: Engineering Data Exchange Format for Use in Industrial Automation Systems Engineering – Automation Markup Language (AutomationML) – Part 1: Architecture and General Requirements; refer to: https://webstore.iec.ch/publication/32339
- IETF RFC 2119: Key Words for Use in RFCs to Indicate Requirement Levels, March 1997; refer to: https://tools.ietf.org/html/rfc2119

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

- ISO/IEC 29500-2: Information Technology – Document Description and Processing Languages – Office Open XML File Formats – Part 2: Open Packaging Conventions
- O-PAS Standard, Version 2.1: Part 2 – Security, The Open Group Preliminary Standard (P210-2), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 4 – O-PAS Connectivity Framework (OCF), The Open Group Preliminary Standard (P210-4), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 6.1 – Information and Exchange Models: Overview and Interfaces, The Open Group Preliminary Standard (P210-6a), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 6.2 – Information and Exchange Models: Basic Configuration, The Open Group Preliminary Standard (P210-6b), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- O-PAS Standard, Version 2.1: Part 6.4 – Information and Exchange Models: Function Blocks, The Open Group Preliminary Standard (P210-6d), May 2021, published by The Open Group; refer to: www.opengroup.org/library/p210
- PLCopen Creating Compliant Function Block Libraries, Version 1.0, PLCopen; refer to: https://plcopen.org/downloads/plcopen-creating-compliant-fb-libraries-version-10
- Unified Architecture (UA) for Programmable Logic Controllers based on IEC 61131-3 (OPC 30000); refer to: https://opcfoundation.org/developer-tools/specifications-opc-ua-information-models/opc-ua-for-plcopen/
- Unified Architecture (UA) Specification (OPC 30081): OPC UA for Process Automation Devices – PA-DIM™, published by the OPC Foundation; refer to: https://reference.opcfoundation.org/v104/PADIM/v100/docs/
- Unified Architecture (UA) Specification, Part 3 (OPC 10000-3): Address Space Model (also IEC 62541-3:2015), published by the OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part3/
- Unified Architecture (UA) Specification, Part 4 (OPC 10000-4): Services (also IEC 62541-4:2015), published by the OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part4/
- Unified Architecture (UA) Specification, Part 5 (OPC 10000-5): Information Model (also IEC 62541-5:2015), published by the OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part5/
- Unified Architecture (UA) Specification, Part 6 (OPC 10000-6): Mappings (also IEC 62541-6:2015), published by the OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part6/
- Unified Architecture (UA) Specification, Part 8 (OPC 10000-8): Data Access (also IEC 62541-8:2015), published by the OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part8/

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

- Unified Architecture (UA) Specification, Part 9 (OPC 10000-9): Alarms and Conditions (also IEC 62541-9:2015), published by the OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part9/
- Unified Architecture (UA) Specification, Part 100 (OPC 10000-100): Devices (also IEC 62541-100), OPC Foundation; refer to: http://www.opcfoundation.org/UA/Part100/

1.5 Terminology

The key words “shall”, “shall not”, and “should” are used in the O-PAS Standard as defined below. These definitions are taken from IETF RFC 2119.

Shall	Definition is an absolute requirement of the specification.
Shall not	Definition is an absolute prohibition of the specification.
Should	There may exist valid reasons in particular circumstances to ignore a particular item, but the full implications must be understood and carefully weighed before choosing a different course.

1.6 Future Directions (Informative)

The models in Part 6.6 are expected to be extended as functionalities and interfaces are defined. In particular, the configuration of OPC UA subscriptions by OCFB_IN is highly simplified in this release. A more detailed mechanism that, among other things, allows the provision of default values instead of configuring these settings on each block individually is planned in a future version of the O-PAS Standard.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

2 Terms and Definitions

For the purposes of the O-PAS Standard, the following terms and definitions apply. Merriam-Webster's Collegiate Dictionary should be referenced for terms not defined in this chapter. The OPAF Glossary and Abbreviations should be referenced for additional terms not defined in this chapter.

Note: This document describes how IEC 61131-3 Programming Languages can be used in the O-PAS Standard. Since IEC 61131-3 is the primary standard of the IEC 61131 family for this purpose, every mention of IEC 61131 without dash should be considered to address IEC 61131-3.

2.1 OPAF Glossary and Abbreviations

For the OPAF Glossary and Abbreviations, see the definitions at: www.opengroup.org/open-process-automation-forum/glossary.

2.2 Terms Specific to Part 6.6

The following terms are specific to O-PAS Part 6.6 (and therefore do not appear in the OPAF Glossary referenced in Section 2.1).

Term	Definition
O-PAS IEC 61131 Application	An O-PAS control application that is programmed using the programming languages and practices defined in IEC 61131-3.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

3 Information Model (Informative)

3.1 Abstract Information Model

The IEC 61131-3 standard uses the model shown in Figure 1. While this figure shows all the relevant information, it contains several ambiguous terms and their meanings may be difficult to interpret in the context of the O-PAS Standard.

The definitions that follow the figure in the IEC 61131-3 standard are also kept rather generic and contain many other definitions that need to be looked up in other parts of the IEC 61131 standard to be understood. Additionally, the IEC 61131-3 standard uses some terms (such as configuration or resource) differently than the O-PAS Standard or not at all (e.g., application).

In the O-PAS Standard one of the choices for implementing Layer-F applications is IEC 61131-3. In the scope of the O-PAS Standard, Layer-F applications implemented in IEC 61131-3 are called “O-PAS IEC 61131 applications” and they are similar to a “configuration” in IEC 61131-3.

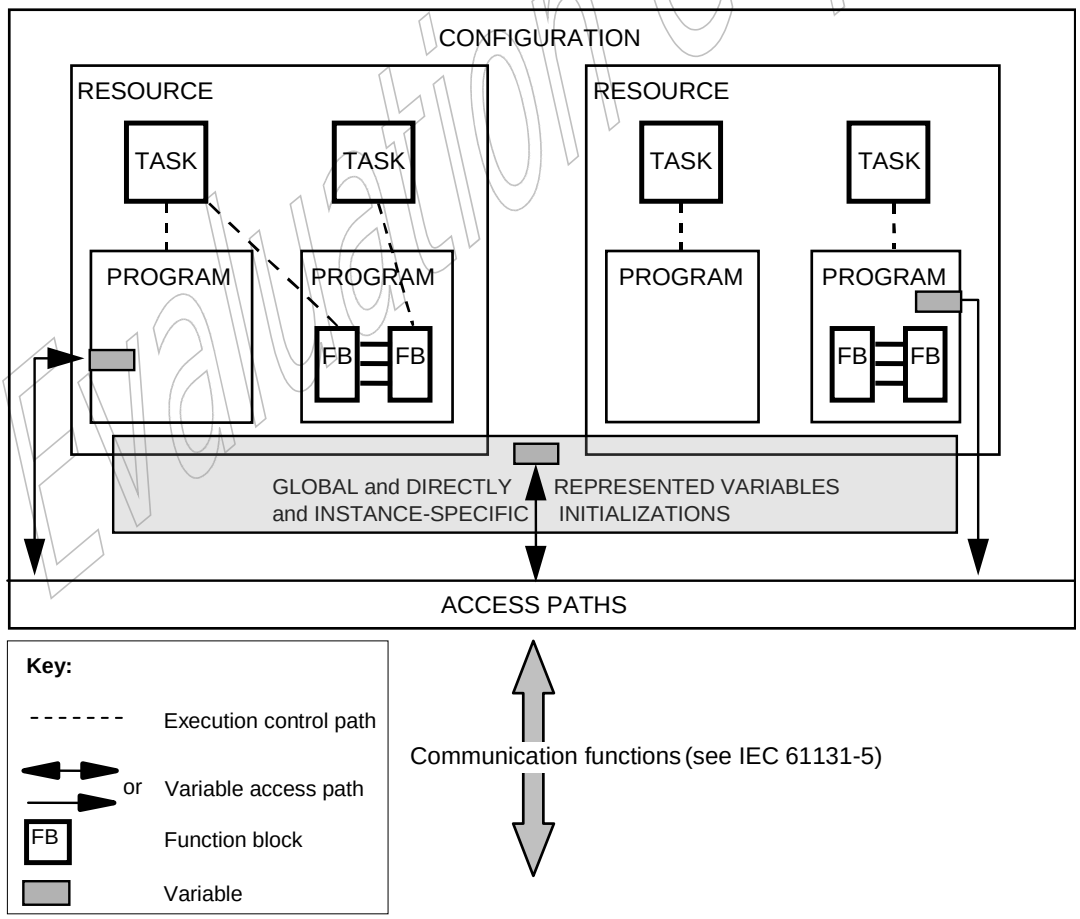


Figure 1: (Informative) IEC 61131-3 Model (from IEC 61131-10)

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

In order to understand this graphic, the XML representation standard for IEC 61131-3, namely IEC 61131-10 which is an evolution of PLCopen XML, is a very useful resource. The definitions of the XML elements for the parts of the figure provide an easy-to-understand breakdown of what each of the parts can contain. These details can be used to map the IEC 61131-3 model (Figure 1) onto the O-PAS object model defined in O-PAS Part 6.2 – Information and Exchange Models: Basic Configuration (Figure 2).

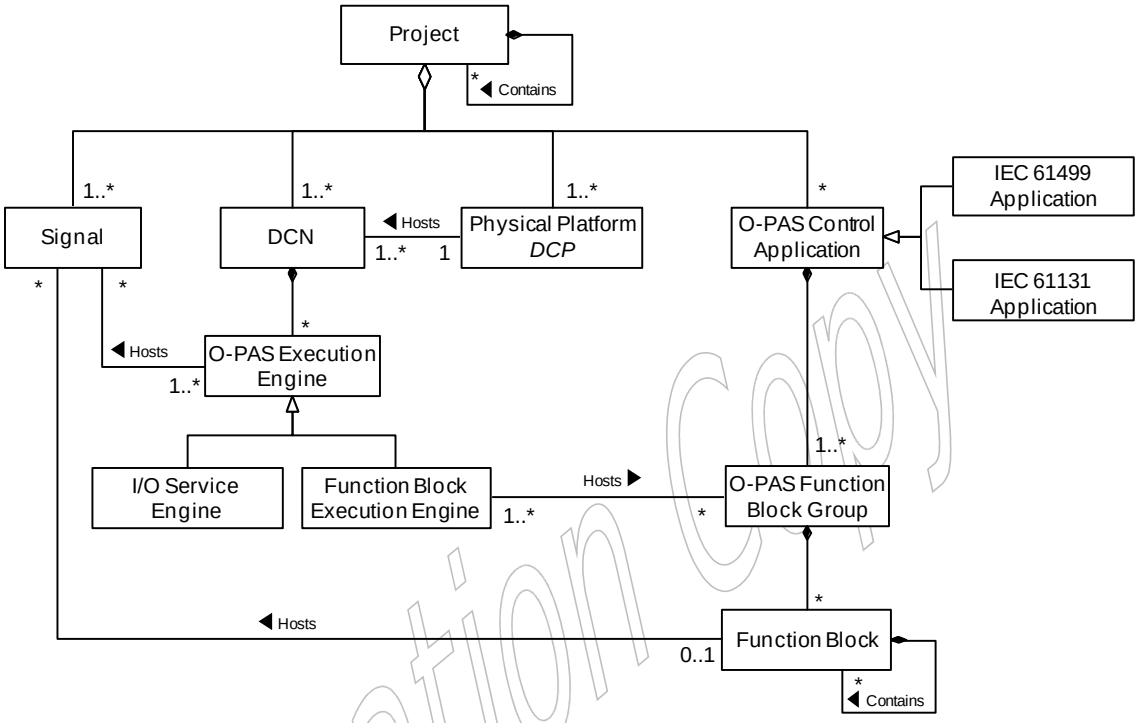


Figure 2: Top-Level O-PAS Object Model (from O-PAS Part 6.2)

The top-level element of IEC 61131-10 XML is the Project (Figure 3) which is what a user commonly opens with IEC 61131-3 editing tools. It fulfills a similar purpose to the project defined in the O-PAS Standard: it groups all the components together. However, it is not part of the IEC 61131-3 standard but rather added by the export structure defined in IEC 61131-10.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

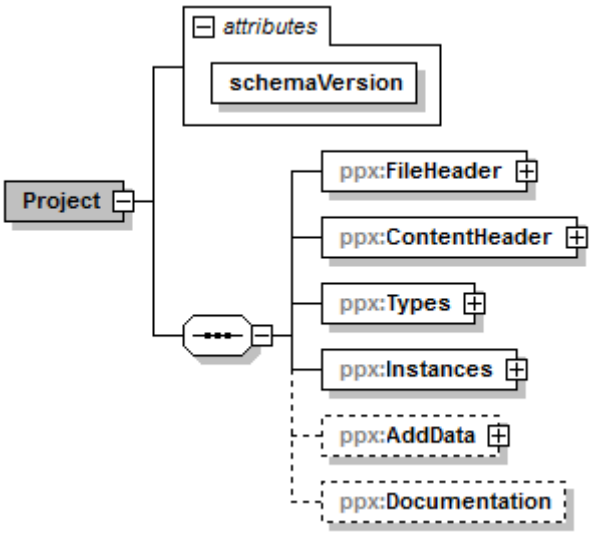


Figure 3: Main Schema Element "Project" in IEC 61131-10 (from IEC 61131-10)

Note: An explanation of the notation can be found at http://taf-jsg.info/wp-content/uploads/2017/11/Annex_6.1_XSD_Diagram_Notations.pdf.

Relevant for the O-PAS Standard is the Types member of the Project (Figure 3) as well as the Instances member (Figure 5). Types (Figure 4) contains definitions for all the Program Organization Units (POUs); i.e. all the programs and function block types, class types, and functions, as well as all user-defined data types.

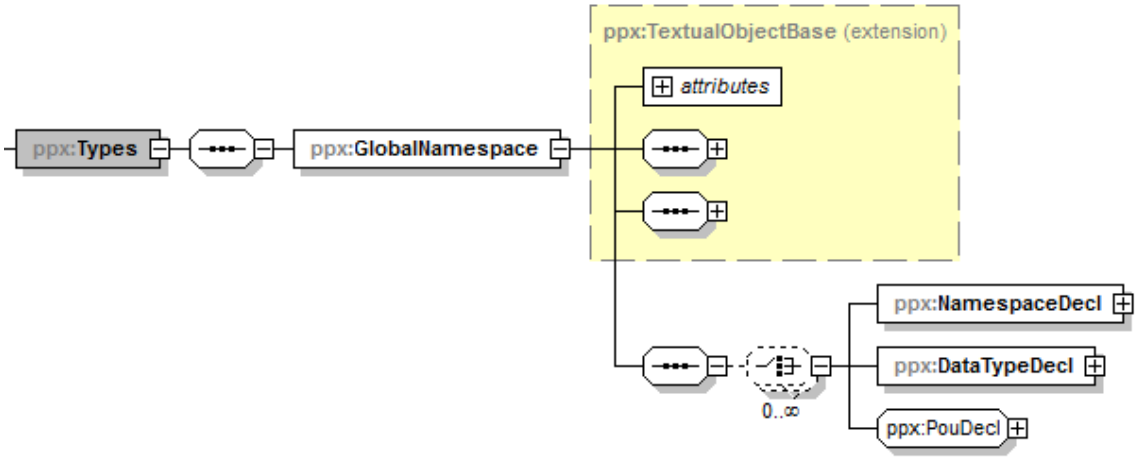


Figure 4: Element "Types" (from IEC 61131-10)

The Instances member (Figure 5) is used to represent the InstanceHierarchy of IEC 61131 projects and is a collection of configurations as defined in IEC 61131-3 and makes use of all the types defined under Types. The O-PAS term equivalent to a configuration is the control application. All control logic is inside a configuration.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

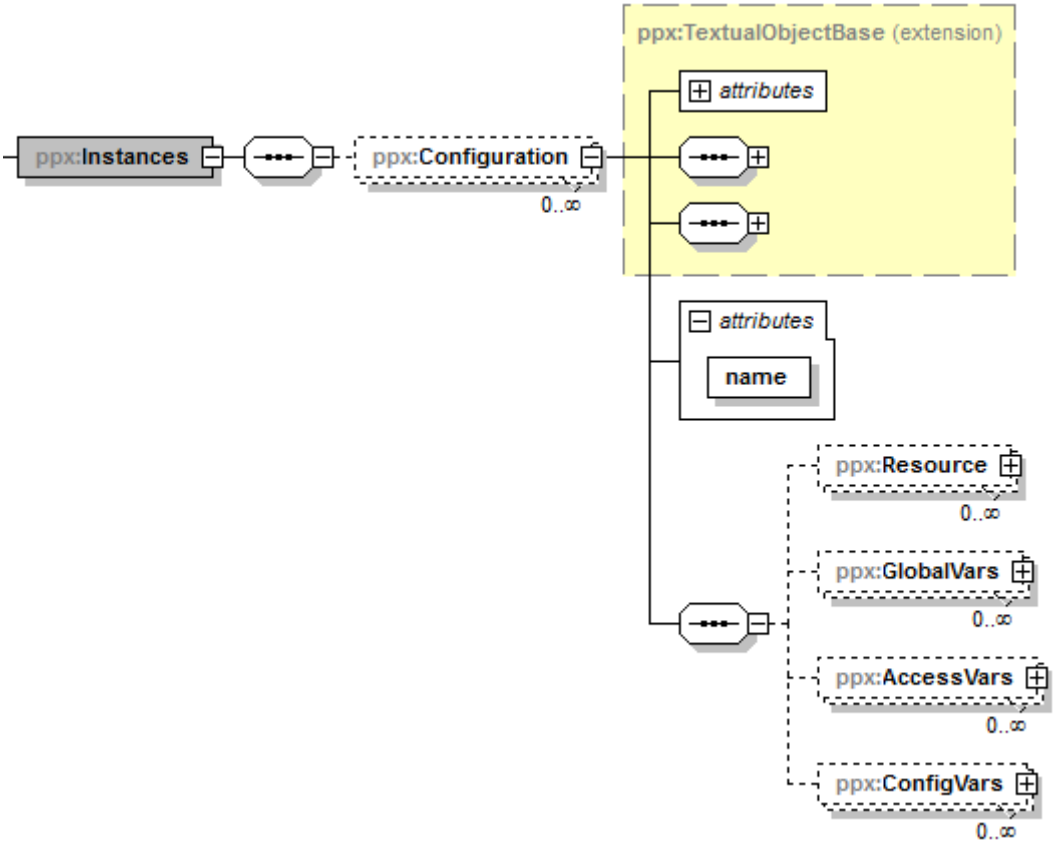


Figure 5: Element "Instances" (from IEC 61131-10)

Each configuration contains the following elements:

- **Resource**: a grouping of POUs and tasks that provides a specific control functionality; details are provided later in this chapter
- **GlobalVars**: variables defined in the configuration (VAR_GLOBAL) that can be accessed from all resources within a configuration (via VAR_EXTERNAL)

  They are only part of the O-PAS model when a POU maps one of the VAR_EXTERNAL variables to an OPC UA variable.
- **AccessVars**: inter-Programmable Logic Controller (PLC) communication as defined in IEC 61131-5

  Must not be used in an O-PAS conformant application (see Section 3.2).
- **ConfigVars**: used to provide memory locations for directly located variables declared with % and the placeholder * (see IEC 61131-3 for details)

  VAR_CONFIG is only relevant for memory management in PLCs – it doesn't actually declare new variables. Although VAR_CONFIG is not relevant to the O-PAS information model, its usage is not restricted.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Figure 6 shows the details of the Resource element. IEC 61131-3 describes a resource as follows:

A resource corresponds to a "signal processing function" and its "man-machine interface" and "sensor and actuator interface" functions (if any) as defined in IEC 61131-1.

This means that the resources are a concept used to split the code into different parts based on the functionality provided by the code; every resource takes care of one control aspect (signal processing function) that the application should fulfil. The O-PAS Standard only looks at the application as a whole or at how the execution of the code might be split across multiple execution engines; it does not address a breakdown of the application based on functionality like the concept of a resource does. Therefore, the IEC 61131 term "resource" does not map directly to any single O-PAS term. In the O-PAS Standard, all resources together form the function block group; the best mapping of the term "resource" is thus "part of a function block group".

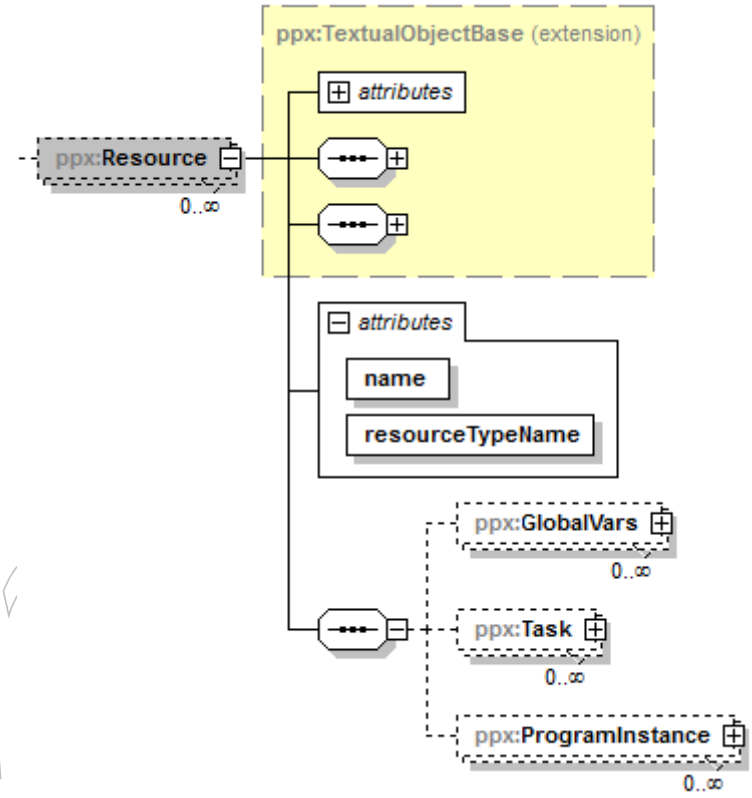


Figure 6: Element "Resource" (from IEC 61131-10)

While a Resource element as a whole cannot be mapped properly, some of the elements it contains are relevant for the O-PAS Standard:

- **GlobalVars**: as defined above
- **Task**: used to schedule execution of POUs, either periodically or non-periodically on a rising edge

  An O-PAS trigger can be used to initiate the execution of a non-periodic task (using the SINGLE input).

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

- **ProgramInstances**: this is where all the top-level POUs are listed that are executed by tasks in this application

  ProgramInstances only instantiates the POUs; their definitions are located in the "Types" element. All logic is contained in POUs and their O-PAS equivalent is the function block. Figure 7 shows the contents of a ProgramInstance.

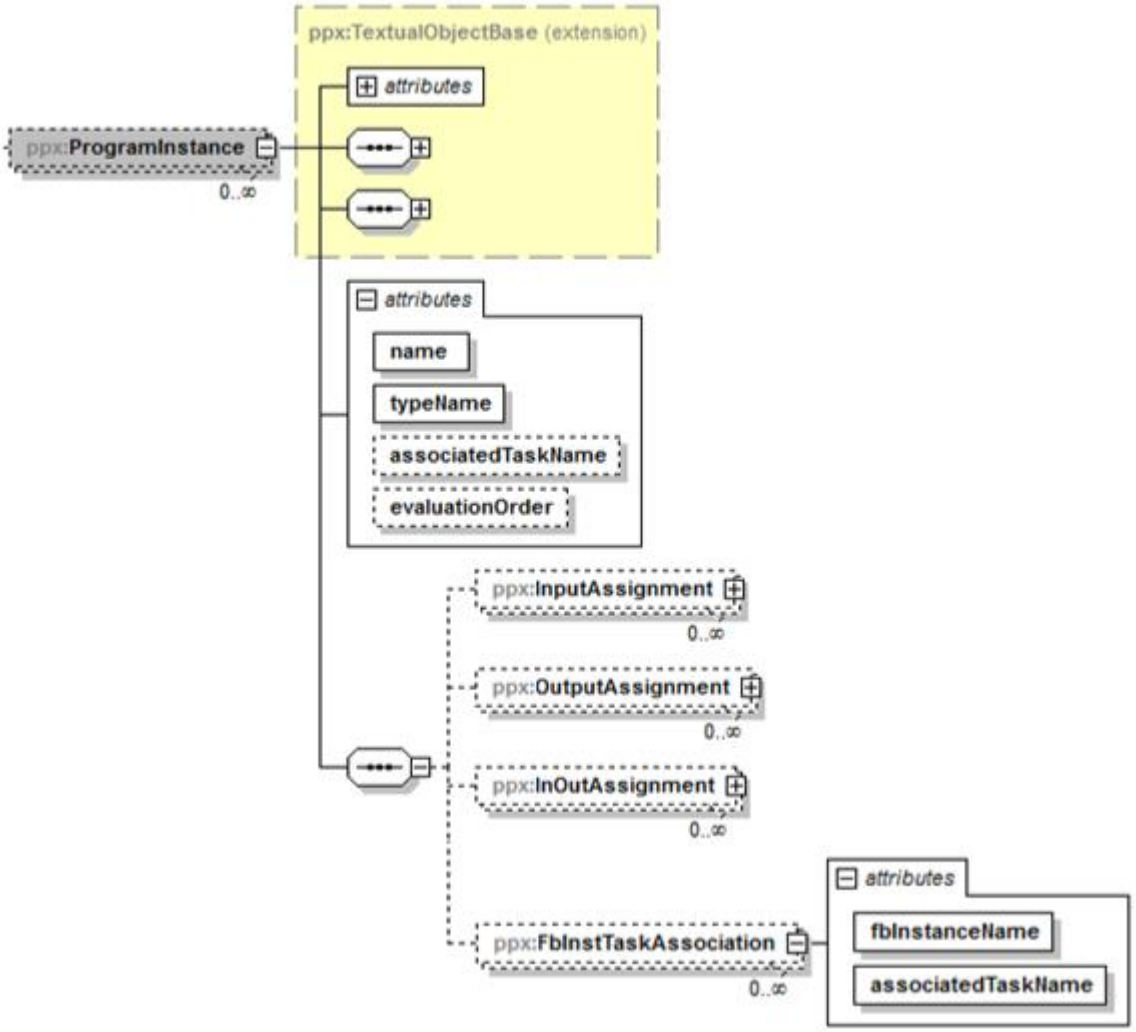


Figure 7: Element "ProgramInstance" (from IEC 61131-10)

ProgramInstances instantiates the types defined in "Types". This element is the transitioning point from the general O-PAS object model (Figure 2) into the function block model (Figure 8).

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

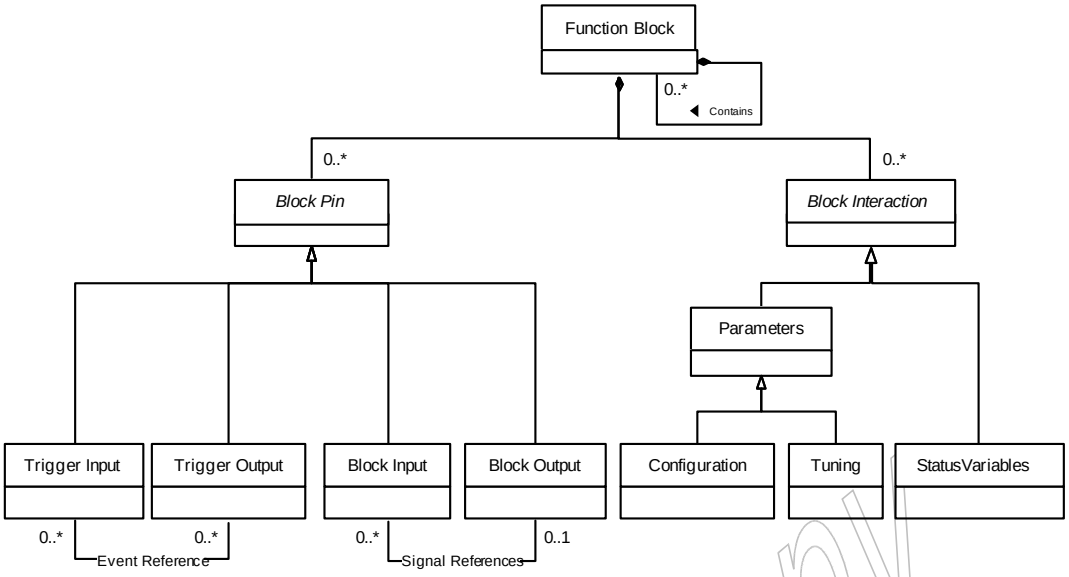


Figure 8: O-PAS Function Block Model

The "Types" (Figure 4) defined in XML are either POU types (PouDecl) or data types (DataTypeDecl). The latter are not relevant for the O-PAS model. There are multiple kinds of PouDecl elements (Figure 9) and some of them have a direct mapping to the O-PAS information model.

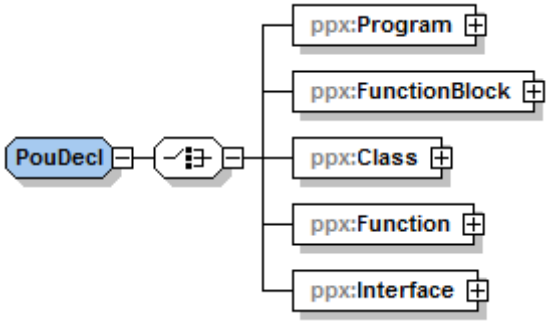


Figure 9: Element "PouDecl" (from IEC 61131-10)

The different kinds of PouDecl elements map to the O-PAS Standard as follows:

- **Program**: maps onto the "function block" from O-PAS Part 6.2

  The O-PAS information model is focused on the modeling of data at runtime, not execution or instantiation of programs, and can therefore treat programs and function blocks in the same way.

- **FunctionBlock**: maps onto the "function block" type from O-PAS Part 6.2

- **Class**: maps onto the "function block" type from O-PAS Part 6.2 – treated like a function block

- **Function**: does not have permanent (non-stack) memory associated with it and is thus not part of the O-PAS information model (which is concerned with data) at this time

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

- **Interface:** does not allow for the definition of variables and is thus not part of the O-PAS data model at this time

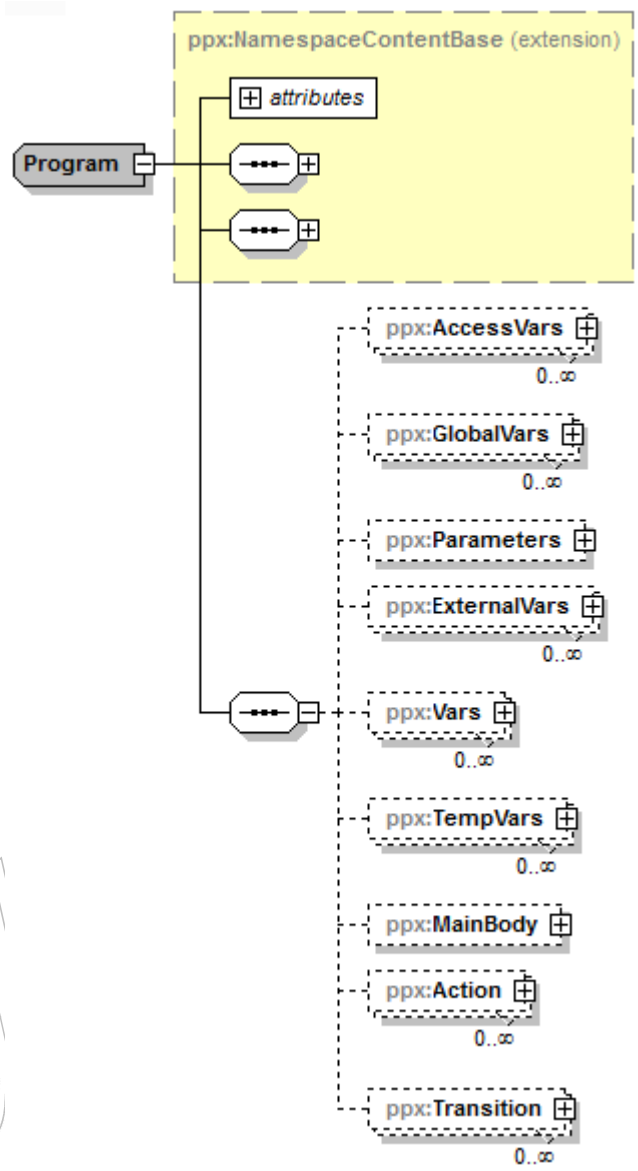


Figure 10: Element "Program Declaration" (from IEC 61131-10)

The contents of a Program element are shown in Figure 10 and map to the O-PAS Standard as follows:

- **AccessVars**: not relevant for the O-PAS Standard
- **GlobalVars**: treated like regular Vars
- **Parameters**: all VAR_INPUT and VAR_OUTPUT of the IEC 61131 program are grouped by the Parameters element

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

They map to the O-PAS block input and O-PAS block output respectively, or trigger inputs and outputs. O-PAS triggers are represented in IEC 61131 as rising edges on a BOOL.

- **ExternalVars**: the O-PAS Standard treats external Vars like regular Vars
- **Vars**: variables in Vars can be mapped to variables in the O-PAS function block model
- **TempVars**: since VAR_TEMP is not permanently in memory (stack-only) it is not mapped to the O-PAS data model
- **MainBody**: this is where the actual code for the program is stored

  The O-PAS information model focuses on data, so this element has no equivalent in the O-PAS information model.
- **Action**: just like MainBody, this element contains code and thus has no equivalent in the O-PAS information model
- **Transition**: transitions are also code and have no equivalent in the O-PAS information model

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

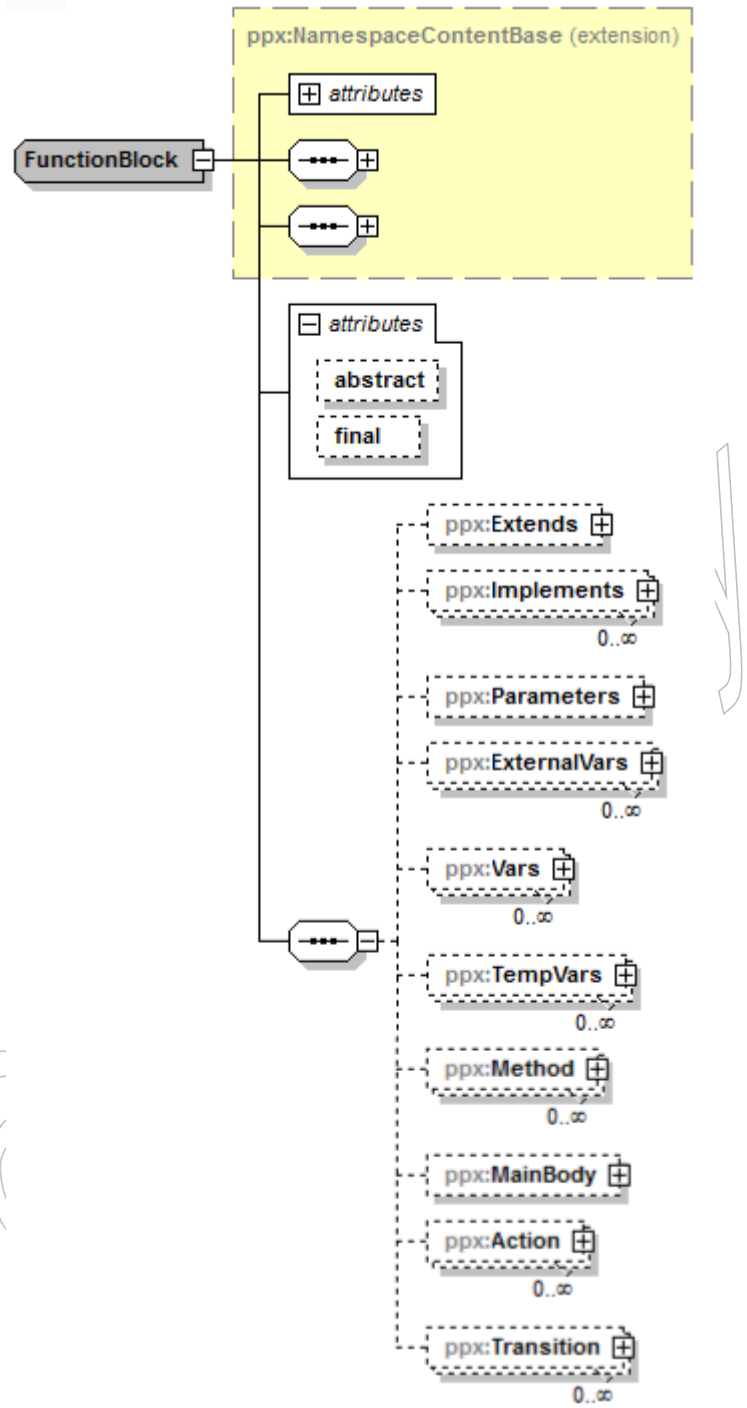


Figure 11: Element "FunctionBlock" (from IEC 61131-10)

The IEC 61131 FunctionBlock element is shown in Figure 11 and maps to the O-PAS Standard as follows:

- **Extends**: used for the inheritance of function blocks

  The O-PAS Standard reflects the possibility that one function block can inherit from another in the OPC UA model as well as the System Unit Classes (SUCs) in the

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Automation Markup Language (AutomationML) standard (IEC 62714-1), also referred to as AML, that represent the block.

- **Implements**: indicates which interfaces this function block implements

  Interfaces don't contain variables and are therefore not relevant to the O-PAS data model.
- **Parameters**: same as for Program
- **Vars**: same as for Program
- **ExternalVars**: same as for Program
- **TempVars**: same as for Program
- **Method**: this element only contains code and no data, and is thus not reflected in the O-PAS information model
- **MainBody**: same as for Program
- **Action**: same as for Program
- **Transition**: same as for Program

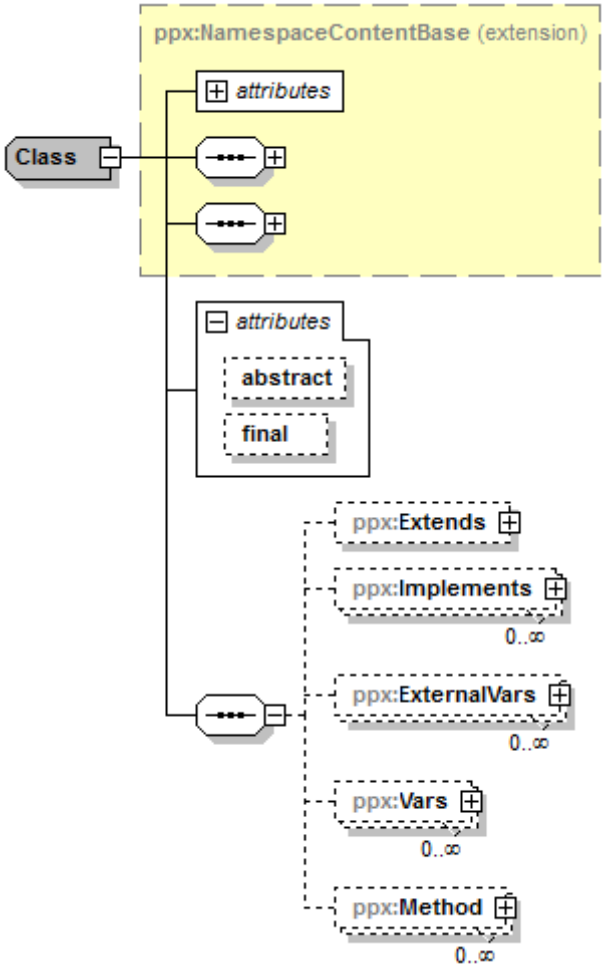


Figure 12: Element "Class" (from IEC 61131-10)

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

The IEC 61131 Class element is shown in Figure 12 and maps to the O-PAS Standard as follows:

- **Extends**: used for the inheritance of classes

  The O-PAS Standard treats classes as function blocks and reflects the possibility that one class can inherit from another in the OPC UA model as well as the SUCs of the AML that represent the block.

- **Implements**: indicates which interfaces this class implements

  Interfaces don't contain variables and are therefore not relevant to the O-PAS data model.

- **Vars**: same as for Program

- **ExternalVars**: same as for Program

- **Method**: this element only contains code and no data, and is thus not reflected in the O-PAS information model

Overall the O-PAS Standard replicates the instance and type hierarchy of the IEC 61131 application in AML as well as OPC UA. However, it simplifies it in that it treats IEC 61131 programs, function blocks, and classes all as O-PAS function blocks. The O-PAS Standard also uses a different variable structure for each function block that is defined in an OPC UA object type – the IEC 61131-3 variables are only part of the O-PAS function block model when mapped to an OPC UA variable by the designer of the block.

A summary of terms and how they map between IEC 61131 and the O-PAS Standard is shown in Table 1. The upper part of the table addresses the general model, the bottom part addresses the function blocks.

Table 1: IEC 61131 (All Subparts) to O-PAS Terminology Mapping

IEC 61131 Term		**IEC 61131-10 Element**	**O-PAS Part 6.2 Term**
Term	**As Defined in Subpart 61131-x**		
General Model			
Project	10	Project	N/A
N/A	N/A	N/A	Project
Configuration	1	Configuration/Instances	O-PAS IEC 61131-3 application (inherits from the O-PAS control application*), and by extension the function block group^
Resource	3	Resource	N/A (Best approximation is part of a function block group/application)

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

IEC 61131 Term		IEC 61131-10 Element	O-PAS Part 6.2 Term
Program	3	Program (linked to a Task with a ProgramInstance)	Function block
FunctionBlock	3	FunctionBlock (possibly linked to a Task with a ProgramInstance)	Function block
Class	3	Class	Function block
FunctionBlocks			
Programmable Controller Hardware	N/A	N/A	DCP
Programmable Controller Firmware that executes Programmable Controller Program	N/A	N/A	DCN and O-PAS execution engine
Variable/Parameter	N/A	VariableDecl	Signal, if mapped to an OPC UA variable
Rising edge on BOOL	3	VariableDecl element with DataType BOOL	Trigger, if mapped to an OPC UA variable
OPAS_XXX_SIGNAL_TYPE STRUCT	3 and this document (Appendix A)	VariableDecl element with correct DataType	Signal, if mapped to an OPC UA variable
VAR, VAR_INPUT, VAR_OUTPUT, VAR_GLOBAL, VAR_EXTERNAL	3	N/A	Signal/trigger, if mapped to an OPC UA variable

* Defined in this document.

^ Since an O-PAS 61131 application can only consist of one function block group (see Section 4.6) by extension the configuration is also mapped onto the function block group.

3.2 IEC 61131-3 Constraints (Informative)

The IEC 61131-3 standard often makes reference to the IEC 61131-5 standard which outlines communication between PLCs. Since the latest version of IEC 61131-5 is from 2000 and does not utilize OPC UA which makes it incompatible with the Open Connectivity Framework (OCF), no aspects of IEC 61131-3 that relate to IEC 61131-5 should be used in an OPAS conformant IEC 61131-3 project.

The nature of IEC 61131-3 is to run on one PLC at a time; spreading execution over multiple devices that are connected on a network (as IEC 61499 does) is not natively supported by IEC

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

61131-3. Therefore, an O-PAS IEC 61131 application only contains one function block group. All function blocks of the application must be in the same function block group. However, there is no limitation on two different O-PAS IEC 61131 applications communicating with each other and working together via the OCF.

The statements provided in this section are informative only and included for completeness and to provide an overview. Section 4.7 provides normative requirements that are connected to these statements.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

4 Import/Export Format

4.1 Import/Export Format Structure

This document uses the XML format defined in the IEC 61131-10 standard (derived from PLCopen XML) for the exchange of PLC application code. All IEC 61131-3 applications that are part of an O-PAS project stored in an Open Packaging Conventions (ISO/IEC 29500-2) file (.opas) shall be stored using this format [CRQ-P6.6-0001]. The integration of the IEC 61131-10 XML into the .opas exchange format is done through the mechanism described in Appendix F of O-PAS Part 6.1 – Information and Exchange Models: Overview and Interfaces.

This mechanism requires a link to the function block types defined in IEC 61131-10 XML (the code file) to OPC UA object types. This is done by adding a custom AddData XML element to the XML elements of type “FunctionBlock” and “Program”. The details for AddData are described in Section 4.3.

Any O-PAS IEC 61131 configuration editing tool that makes changes to the AML, IEC 61131-10 XML, or OPC UA nodeset file shall keep the other two in sync [CRQ-P6.6-0002].

Generic O-PAS configuration editing tools unable to do so do not edit the XML, the nodeset, or the IEC 61131-specific sections of the AML. They are restricted to creating mirrors (see IEC 62714-1) of IEC 61131-FunctionBlockGroups to deploy them to an IEC 61131-Engine or creating internal links (see IEC 62714-1), as described in O-PAS Parts 6.1 and 6.2.

4.2 Open Packaging Conventions

IEC 61131-10 XML files that are part of a .opas package as defined in O-PAS Part 6.1 shall use the Content type “model/vnd.opas.61131-10+xml” and Relationship type https://opengroup.org/open-process-automation/O-PAS/relationships/IEC61131-10 [CRQ-P6.6-0003].

4.3 IEC 61131-10 AddData

For the O-PAS Standard, two different AddData elements are needed: first, a link from program, function block, and class elements to OPC UA object types; and second, a link from individual variables inside that function block/program to OPC UA variables that are part of the corresponding OPC UA object type.

For an example, see Appendix F.

4.3.1 UAObjectTypeLink

The complex type “UAObjectTypeLink” shown in Figure 13 shall be used in the AddData element of each program, function block, or class element in IEC 61131-10 XML [CRQ-P6.6-0004]. It represents the link of that program, function block, or class to the OPC UA object type

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

that is used to display the program, function block, or class in OPC UA. There shall be exactly one UAObjectTypeLink element per AddData element.

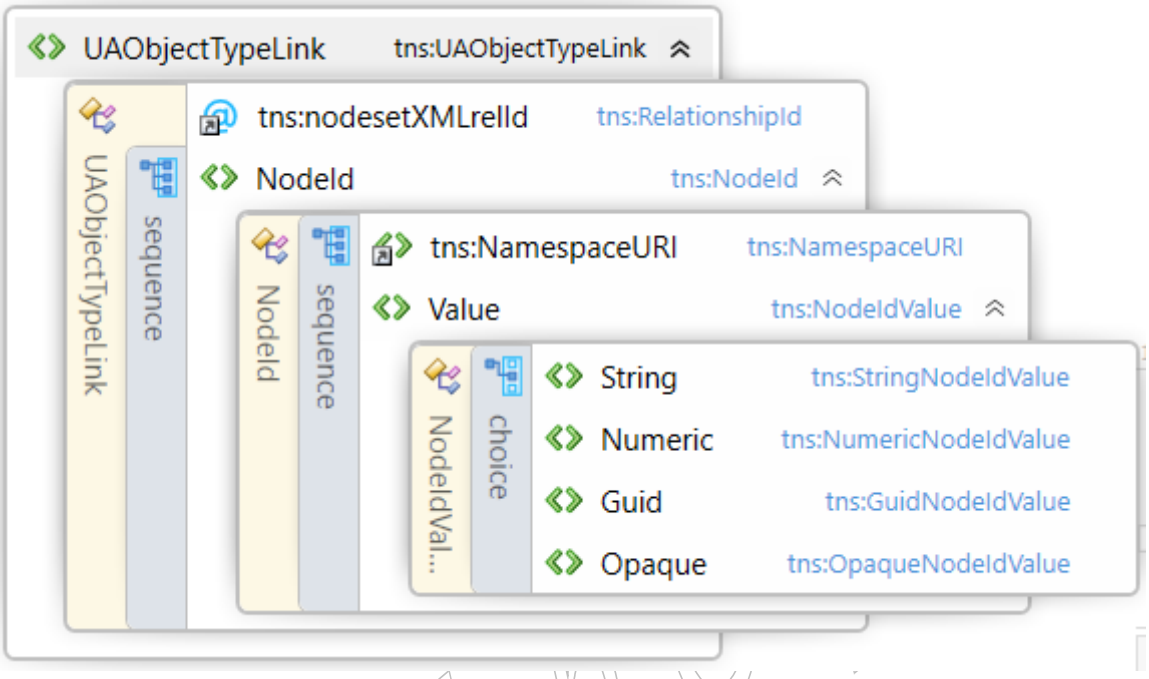


Figure 13: AddData Element UAObjectTypeLink (Drawn with Microsoft® Visual Studio®)

"nodesetXMLrelId" refers to the relationship Id of an OPC relationship from IEC 61131-10 XML. The relationship is from the part that is the IEC 61131-10 XML to a Nodeset2.xml file.

"NodeId" refers to a NodeId in the Nodeset2.xml file referenced by nodesetXMLrelId. The NodeId uses the full NamespaceUri instead of the Namespace Index. The NodeId shall be for an OPC UA object type that is a sub-type (direct or indirect) of 4:FunctionBlockType defined in O-PAS Part 6.2 [CRQ-P6.6-0005]. Additional rules regarding the OPC UA object type are specified in Section 6.2.

If the program, function block, or class element contains instance declarations for any other function blocks/classes as VAR, VAR_INPUT, VAR_OUTPUT, or VAR_GLOBAL, then the UA object type linked shall include HasComponent relationships to UA object instance declarations for each of the declared function blocks or classes. The object types for those instances shall be the ones linked to those IEC 61131 function blocks/classes in their AddData. The BrowseName shall be the same as the instance name for the IEC 61131 declaration and the BrowseName's namespace shall be the same as the one of the object type node that is the origin of the HasComponent relationship [CRQ-P6.6-0006].

4.3.2 UAObjectVariableLink

The complex type "UAObjectVariableLink" shown in Figure 14 shall be used in the AddData of VariableDecl elements that are part of program, function block, or class elements. The complex type is used to link the VariableDecl elements to a UA variable. The variable shall be part of the UA object type to which the function block/program is linked using UAObjectTypeLink [CRQ-P6.6-0007]. This linking is done using a BrowsePath from the UA object type to the variable.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

The linking shall follow the rules described in Section 6.5 for which the IEC 61131-3 data type relates to which OPC UA variable type and OPC UA data type [CRQ-P6.6-0008].

The usage of a UAObjectVariableLink element in an AddData element shall be mutually-exclusive to the usage of a UAObjectVariableAttributeLink element [CRQ-P6.6-0124].

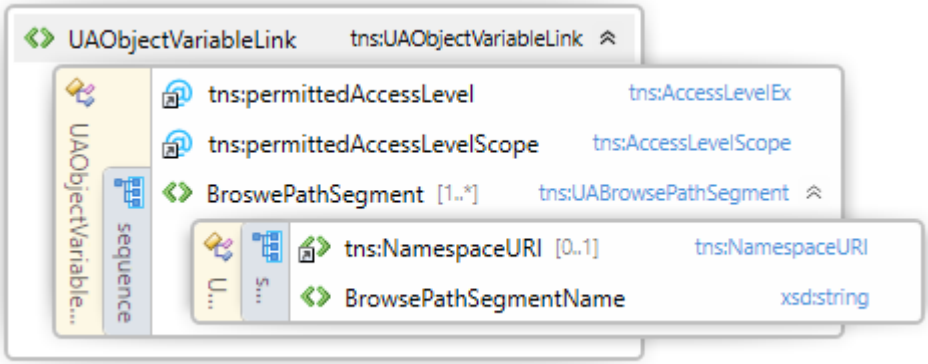


Figure 14: AddData Element UAObjectVariableLink

"BrowsePathSegment" defines the BrowsePath using BrowseNames and NamespaceUris. Note that the BrowsePath can only be forward.

The attribute "permittedAccessLevel" defines the AccessLevelEx to be used for instances of the respective OPC UA variables. See Section 6.5.2 for more details. The values for the attribute shall be the same as the bitmap defined for the OPC UA data type "AccessLevelExType" in OPC UA Part 3.

The attribute "permittedAccessLevelScope" defines to which OPC UA variables the permittedAccessLevel applies. See Section 6.5.2 for more details. The values for the attribute are as follows:

- **VariableOnly**: permittedAccessLevel only applies to the OPC UA variable targeted by the UAObjectVariableLink itself
- **VariableAndProperties**: permittedAccessLevel applies to the OPC UA variable targeted by the UAObjectVariableLink itself and all UA variables connected to it via OPC UA HasProperty references

4.3.3 UAObjectVariableAttributeLink

The complex type "UAObjectVariableAttributeLink" shown in Figure 15 shall be used in the AddData of VariableDecl elements that are part of program, function block, or class elements. The complex type is used to link the VariableDecl elements to a UA attribute of a UA variable. The variable shall be part of the UA object type that the function block/program is linked to using UAObjectTypeLink [CRQ-P6.6-0125]. This linking is done using a BrowsePath from the UA object type to the variable.

The linking shall follow the rules described in Section 6.5 for which the IEC 61131-3 data type relates to which OPC UA variable type and OPC UA data type [CRQ-P6.6-0126].

The usage of a UAObjectVariableAttributeLink element in an AddData element shall be mutually-exclusive to the usage of a UAObjectVariableLink element [CRQ-P6.6-0127].

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

If a UA variable is already linked using UAObjectVariableLink, any links of individual UA attributes using UAObjectVariableAttributeLink shall be dominant and be used when running the function block in a DCN [CRQ-P6.6-0128].

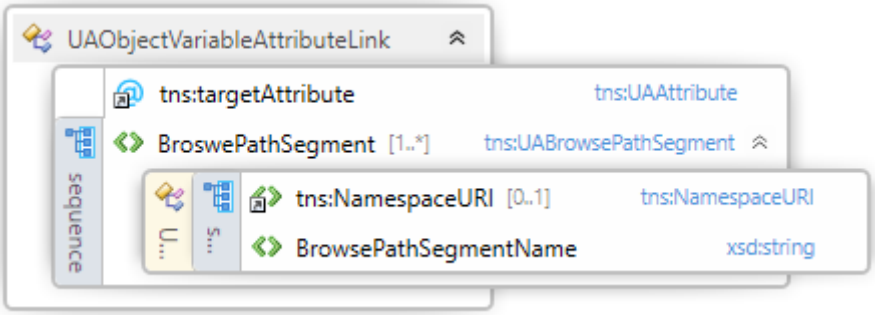


Figure 15: AddData Element UAObjectVariableAttributeLink

"BrowsePathSegment" defines the BrowsePath using BrowseNames and NamespaceUris. Note that the BrowsePath can only be forward.

The attribute "targetAttribute" defines which OPC UA attribute of the UA variable pointed to by the BrowsePathSegments is being linked with the IEC 61131 variable this AddData is for. See Section 6.5.2 for more details regarding the type requirements on the linkage. Not all UA attributes can be linked; only those that make sense for function blocks to change at runtime. The allowed values are as follows:

- **Value_value**: the IEC 61131 variable is linked to the Value element in the DataValue structure of the Value attribute
- **Value_sourceTimestamp**: The IEC 61131 variable is linked to the sourceTimestamp element in the DataValue structure of the Value attribute
- **Value_statusCode**: The IEC 61131 variable is linked to the StatusCode element in the DataValue structure of the Value attribute
- **AccessLevel**: The IEC 61131 variable is linked to the AccessLevel attribute
- **AccessLevelEx**: The IEC 61131 variable is linked to the AccessLevelEx attribute – this attribute linking shall overwrite any AccessLevel links [CRQ-P6.6-0129]

4.4 OPC UA Nodeset file

Programs, function blocks, and classes in an IEC 61131-10 XML file can be linked to OPC UA object types. Those UA object types are defined in a Nodeset2.xml file. For all such links, there shall be Open Packaging Conventions relationships from the IEC 61131-10 XML part to the Nodeset2.xml part. The relationships may be external. They shall use the RelationshipType "https://opengroup.org/open-process-automation/O-PAS/relationships/FB-UAObjecTypeDefinition" [CRQ-P6.6-0009].

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

4.5 AutomationML

In the AML file of a .opas package (see O-PAS Parts 6.1 and 6.2 and IEC 62714-1) it is often needed to link a function block or application to the file that contains its "LogicSource". This is done using the 61131-XMLFileRelationship interface class described in Section 4.5.1. It inherits from the Basic-CodeFileRelationship interface class and is used to indicate that the referenced file is an IEC 61131-10 XML file.

4.5.1 61131-XMLFileRelationship Interface Class

The 61131-XMLFileRelationship interface class inherits from Basic-CodeFileRelationship and shall be used in place of it whenever the referenced relationship is for an IEC 61131-10 XML file [CRQ-P6.6-0010].

For attributes and the formal definition, see Appendix D.

4.5.2 61131-Engine Role Class

The 61131-Engine role class inherits from Basic-FBExecutionEngine and shall be used for execution engines that can execute IEC 61131-3 applications. The mirrors it contains for function block groups shall have the role requirement 61131-FunctionBlockGroup [CRQ-P6.6-0011].

The primary purpose of this role is to identify and distinguish IEC 61131-3 applications.

For attributes and the formal definition, see Appendix D.

4.5.3 61131-Application Role Class

The 61131-Application role class inherits from Basic-FunctionBlockApplication and shall be used for applications that are implemented in IEC 61131-3. The "LogicSource" external interface contained by the role shall use the 61131-XMLFileRelationship interface class (inherits from Basic-CodeFileRelationship). Because IEC 61131-3 does not allow execution spread across multiple devices, there shall also only be one function block group as an internal element and it shall have the role requirement 61131-FunctionBlockGroup [CRQ-P6.6-0012].

For attributes and the formal definition, see Appendix D.

4.5.4 61131-FunctionBlockGroup Role Class

The 61131-FunctionBlockGroup role class inherits from Basic-FunctionBlockGroup and shall be used for function block groups that use IEC 61131-3 [CRQ-P6.6-0013].

For attributes and the formal definition, see Appendix D.

4.5.5 System Unit Classes

For each program, function block, or class type defined in an IEC 61131-10 XML file that is part of a .opas package, an SUC shall be created that corresponds to the type. These SUCs shall be linked to the IEC 61131-10 file using the LogicSource ExternalInterface using the 61131-XMLFileRelationship interface class, and shall link to an OPC UA object type using the UATypeLinkInterface external interface. The OPC UA object type shall be the same as the UA

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

object type linked in the AddData element of the program, function block, or class definition in the XML [CRQ-P6.6-0014].

The SUCs can contain internal elements that reflect the variable structure of the OPC UA object type and allow configuring OPC UA variables of the type which are not connected to variables in the code. If implemented, this shall be done as described in Appendix E of O-PAS Part 6.1 [CRQ-P6.6-0015].

If the program, function block, or class element represented by the SUC contains instance declarations for any other function blocks or classes as VAR, VAR_INPUT, VAR_OUTPUT, or VAR_GLOBAL, then the SUC shall include internal elements for each of the declared function blocks/classes. The internal elements shall be of the SUCs corresponding to the IEC 61131 function blocks/classes and names of the internal elements shall be the same as the instance name for the IEC 61131 declaration. The internal elements shall also have the role requirement UA-NodeRelative [CRQ-P6.6-0016].

4.5.6 InstanceHierarchy

In the InstanceHierarchy of an AML file there shall be an element with the role requirement 61131-Application for each IEC 61131-10 application that is part of the project. This element shall be linked to an IEC 61131-10 XML file that is in the .opas package using the LogicSource interface [CRQ-P6.6-0017].

The application AML element in the InstanceHierarchy shall then contain one internal element with the role requirement 61131-FunctionBlockGroup [CRQ-P6.6-0018].

The function block group AML element shall then contain internal elements for ProgramInstances in the linked IEC 61131-10 XML file. The internal elements shall be of the SUC that corresponds to the ProgramInstance's IEC 61131 type [CRQ-P6.6-0019]. Through the internal elements in the SUCs this effectively creates an AML element in the InstanceHierarchy for every function block instance in the IEC 61131 application and thus for every OPC UA object instantiated in a UA server for the application.

For IEC 61131-3 execution engines that are hosted by a DCN there shall be an internal element to the DCN element with the role requirement 61131-Engine. This element shall then contain mirrors of the 61131-FunctionBlockGroup elements of the applications that are executed by this engine [CRQ-P6.6-0020].

Each 61131-FunctionBlockGroup element shall only have one AML mirror in 61131-Engine elements; i.e., each function block group shall only be deployed to one execution engine [CRQ-P6.6-0130].

4.6 Import/Export Constraints

In accordance with Section 3.2, none of the XML elements regarding IEC 61131-5 shall be included in an IEC 61131-10 XML file that is part of an O-PAS configuration exchange file [CRQ-P6.6-0021]. These aspects of IEC 61131-10 are not required to be O-PAS conformant.

The Instances member of the Project element of the XML shall only contain one configuration element [CRQ-P6.6-0022]. This is because when loaded into DCNs, data cannot be shared beyond configurations within the same XML file without using the OCF, thus it must be listed in

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

the AML as two separate applications, each with their own XML, to allow other tools to connect signals correctly.

Other than the O-PAS defined AddData elements, the AddData of an IEC 61131-10 XML file shall only contain elements that can be removed without negatively affecting the application [CRQ-P6.6-0023]. This means that all non-conformant O-PAS AddData elements can be removed and the application can still be executed.

4.7 Import/Export Profiles, Facets, and Conformance Units

4.7.1 Profiles

There is one profile defined for IEC-61131-conformant configuration tools:

- CTL-601 IEC 61131 Import/Export: this profile includes all of the conformance units defined in this chapter and the XML schema defined in Appendix C

  In addition, it depends on:

  — SEC-F-001 defined in O-PAS Part 2 [CRQ-P6.6-0133]

  — OBC-201 defined in O-PAS Part 6.2 [CRQ-P6.6-0134]

  — OFB-002 defined in O-PAS Part 6.4 [CRQ-P6.6-0135]

4.7.2 Conformance Units

For conformance units, see the statements in this chapter marked with “CRQ” identifiers.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

5 Download/Upload Exchange Format

Since PLCs are highly different in their nature it is difficult to find a standardized format that can be directly executed by various PLCs of different sizes or CPU architectures. Therefore, the download/upload format for IEC 61131-3 as described in Chapter 5 of O-PAS Part 6.1 – Information and Exchange Models: Overview and Interfaces is left to each vendor in this version of the O-PAS Standard.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

6 OCF Information Model

An O-PAS IEC 61131 application replicates its internal InstanceHierarchy of POUs in its OPC UA nodeset. This makes it easy to identify which UA object in the nodeset is connected to which POU in the IEC 61131 application. Additionally, it makes it possible to identify relations amongst the UA variables; for example, all variables making up a PID are located in the same object.

The OPC UA nodeset also reflects the type hierarchy of the IEC 61131 function block application. Inheritance via the Extends keyword on function blocks is reflected in the UA nodeset through inheritance of the corresponding OPC UA object types.

6.1 O-PAS OPC UA Object Types

The following sections define the O-PAS 61131 OPC UA object types. For definitions of OPC UA Device Integration, OPC UA PA-DIM™, and OPC UA-based objects, refer to the corresponding standards.

For the O-PAS types that the types in this section inherit from, refer to O-PAS Part 6.2.

An overview of the most important object types and their usage is shown in Figure 16.

Implementations shall ensure adherence to Mandatory (M) aspects in order to comply with the O-PAS Standard. Optional [O] aspects may or may not be implemented within a project.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

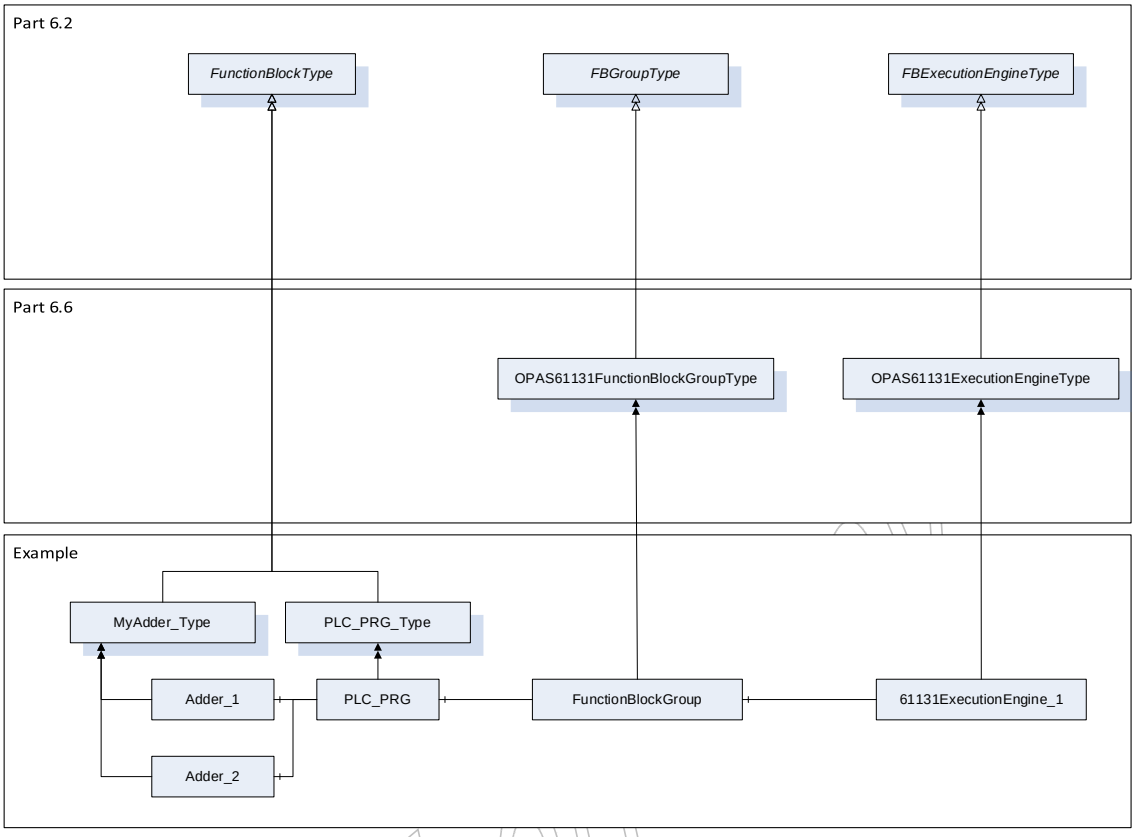


Figure 16: Overview and Example of UA Object Types

6.1.1 OPAS61131ExecutionEngineType

Instances of this type are used to represent execution engines within a DCN that execute O-PAS IEC 61131-3 applications. The OPAS61131ExecutionEngineType defined in this document is a sub-class of the abstract type defined in O-PAS Part 6.2. Table 2 details the properties for the O-PAS61131ExecutionEngineType.

Table 2: O-PAS61131ExecutionEngineType

Attribute	**Value**				
BrowseName	OPAS61131ExecutionEngineType				
IsAbstract	FALSE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of 4:FBExecutionEngineType defined in O-PAS Part 6.2.					
0:HasComponent	Object	<61131 Function Block Groups>		OPAS61131FunctionBlockGroupType	Mandatory Placeholder

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

6.1.2 OPAS61131FunctionBlockGroupType

Each O-PAS 61131 function block group within an O-PAS 61131 execution engine represents a complete O-PAS IEC 61131-3 application, the equivalent of a “configuration” in the IEC 61131-3 standard. Table 3 details the properties for the O-PAS61131FunctionBlockGroupType.

Table 3: OPAS61131FunctionBlockGroupType

Attribute	**Value**				
BrowseName	OPAS61131FunctionBlockGroupType				
IsAbstract	FALSE				
References	**NodeClass**	**BrowseName**	**DataType**	**TypeDefinition**	**Modeling Rule**
Sub-type of 4:FBGroupType defined in O-PAS Part 6.2.					
0:HasComponent	Object	<FunctionBlocks>		4:FunctionBlock Type	Optional Placeholder

Top-level POUs (programs, function blocks, and classes) of an O-PAS IEC 61131-3 application are instantiated through ProgramInstances in the resources of the configuration for the application. OPC UA objects representing these top-level POU instances shall be used to fill the FunctionBlocks placeholder [CRQ-P6.6-0024].

6.2 Type AddressSpace

For the purposes of this document, 4:FunctionBlockType defined in O-PAS Part 6.2 is treated as abstract.

In order to display an IEC 61131 program, function block, or class in OPC UA it must be mapped to an OPC UA object type that is a sub-type (direct or indirect) of 4:FunctionBlockType. See Section 4.3 for details on how this mapping works.

These OPC UA object types can either be created by the user themselves or be part of a standardized OPC UA namespace such as the one created for O-PAS Part 4 – O-PAS Connectivity Framework (OCF). Tools can also automatically create UA object types for IEC 61131-3 programs, function blocks, and classes. At runtime all OPC UA object types mapped to programs, function blocks, and classes used in an O-PAS IEC 61131 application shall be present in the type hierarchy of AddressSpace of the execution engine executing the application [CRQ-P6.6-0110].

6.2.1 Rules for OPC UA Object Types for IEC 61131-3 Programs, Function Blocks, and Classes

If an IEC 61131-3 function block/class extends (inherits from) another existing function block/class, the corresponding OPC UA object types shall reflect this inheritance via sub-typing [CRQ-P6.6-0025]; i.e., the OPC UA object type for the child function block/class type is a sub-type of the OPC UA object type of the parent function block/class type, not FunctionBlockType.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

All OPC UA object types for function block/class types still have FunctionBlockType at some level in their type inheritance.

If a function block/class declares instances of other function blocks/classes in its VAR, VAR_INPUT, VAR_OUTPUT, or VAR_GLOBAL sections, then the OPC UA object type for the function block/class shall contain OPC UA instance declarations using the respective OPC UA object type for the declared instance of the other function block/class and a HasComponent relationship [CRQ-P6.6-0026]. An example is shown in Figure 16 where PLC_PRG contains two instances of a function block type MyAdder called Adder_1 and Adder_2.

Individual UA variables in the UA object types may be linked to variables declared in the VAR, VAR_INPUT, VAR_OUTPUT, VAR_GLOBAL, or VAR_EXTERNAL sections of a program, function block, or class. If such a link is defined, the value of the IEC 61131-3 variable shall be used as the value of the UA variable [CRQ-P6.6-0027]. See Section 4.3.2 for how the linking is expressed in IEC 61131-10 XML. The number, data types, and variable types of the OPC UA variables in the UA object type are not restricted. There can be variables in the OPC UA that are not linked in IEC 61131-10 XML. If the AML file defines instance-specific values for these variables, then those values shall be used for the instances of the object type; see Section 4.5 for details on the AML [CRQ-P6.6-0028].

6.3 Instance AddressSpace

The DCNBaseType object contains all the OPAS61131ExecutionEngineType objects which in turn contain all the OPAS61131FunctionBlockGroupType objects.

For every O-PAS IEC 61131 application that is being executed in an execution engine, the execution engine's OPC UA object shall have as a component an instance of OPAS61131FunctionBlockGroupType which in turn shall have components for the POUs of the application [CRQ-P6.6-0029].

If configured in AML via internal links in the InstanceHierarchy, OPC UA variables created for instances of O-PAS Connectivity Function Blocks (OCFBs) as defined in Section 7.2 shall register an alias to the Tags or Topics alias name categories (refer to O-PAS Part 4 for more information on alias) [CRQ-P6.6-0030].

If configured in AML via internal links in the InstanceHierarchy, OPC UA objects created for instances of OCFBs as defined in Section 7.2 shall register an alias to the FunctionBlocks alias name category (see O-PAS Part 6.2) [CRQ-P6.6-0031].

When it can be determined from the code, the OPC UA variables representing the VAR_INPUT variables of one function block shall have an OPC UA reference of type 4:ConnectedToOutput to the OPC UA variable that represents the VAR_OUTPUT to which the VAR_INPUT is connected; e.g., through a wire in a ladder diagram [CRQ-P6.6-0032].

6.4 Namespaces

6.4.1 Namespace Metadata

Table 4 defines the namespace metadata for this document. The object is used to provide version information for the namespace and an indication about static nodes. Static nodes are identical for all attributes in all servers, including the Value attribute. See OPC UA Part 5 for more details.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

The information is provided as an object of type NamespaceMetadataType. This object is a component of the namespaces object that is part of the server object. The NamespaceMetadataType object type and its properties are defined in OPC UA Part 5.

The version information is also provided as part of the ModelTableEntry in the UANodeSet XML file. The UANodeSet XML schema is defined in OPC UA Part 6.

Table 4: Namespace Metadata Object for this Document (O-PAS Part 6.6)

Attribute	**Value**		
BrowseName	https://www.opengroup.org/open-process-automation/O-PAS/61131		
References	**BrowseName**	**DataType**	**Value**
0:HasProperty	NamespaceUri	String	https://www.opengroup.org/open-process-automation/O-PAS/61131
0:HasProperty	NamespaceVersion	String	2.1
0:HasProperty	NamespacePublicationDate	DateTime	2021-05-17
0:HasProperty	IsNamespaceSubset	Boolean	True
0:HasProperty	StaticNodeIdTypes	IdType []	Integer
0:HasProperty	StaticNumericNodeIdRange	NumericRange []	0,7000
0:HasProperty	StaticStringNodeIdPattern	String	Null

6.4.2 Handling of OPC UA Namespaces

Namespaces are used by the OPC UA to create unique identifiers across different naming authorities. The attributes NodeId and BrowseName are identifiers. A node in the UA AddressSpace is unambiguously identified using a NodeId. Unlike NodeIds, the BrowseName cannot be used to unambiguously identify a node. Different nodes may have the same BrowseName. They are used to build a BrowsePath between two nodes or to define a standard property.

Servers may often choose to use the same namespace for the NodeId and the BrowseName. However, if they want to provide a standard property, its BrowseName shall have the namespace of the standards body although the namespace of the NodeId reflects something else; for example, the EngineeringUnits property. All NodeIds of nodes not defined in this document shall not use the standard namespaces [CRQ-P6.6-0033].

Table 5 provides a list of mandatory and optional namespaces used in an O-PAS OPC UA server.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Table 5: Namespaces Used in an O-PAS Server

NamespaceURI	Description	Use
http://opcfoundation.org/UA/	Namespace for NodeIds and BrowseNames defined in the OPC UA specification. This namespace shall have namespace index 0.	Mandatory
Local Server URI	Namespace for nodes defined in the local server. This namespace shall have namespace index 1.	Mandatory
http://opcfoundation.org/UA/DI/	Namespace for NodeIds and BrowseNames defined in OPC UA Part 100. The namespace index is server-specific.	Mandatory
http://opcfoundation.org/UA/Dictionary/IRDI	Namespace for NodeIds of International Registration Data Identifiers (IRDIs) using HasDictionaryEntry. The namespace index is server-specific.	Optional
http://opcfoundation.org/UA/PADIM/	Namespace for NodeIds and BrowseNames defined in the OPC 30081 PA-DIM specification. The namespace index is server-specific.	Mandatory
https://www.opengroup.org/open-process-automation/O-PAS/Base	Namespace for NodeIds and BrowseNames defined in O-PAS Part 6.2. The namespace index is server-specific.	Mandatory
https://www.opengroup.org/open-process-automation/O-PAS/FunctionBlocks	Namespace for NodeIds and BrowseNames defined in O-PAS Part 6.4. The namespace index is server-specific.	Mandatory
https://www.opengroup.org/open-process-automation/O-PAS/61131	Namespace for NodeIds and BrowseNames defined in this document. The namespace index is server-specific.	Mandatory
Vendor-specific types	A server may provide vendor-specific types like types derived from object types defined in this document in a vendor-specific namespace.	Optional
Vendor-specific instances	A server provides vendor-specific instances of the standard types or vendor-specific instances of vendor-specific types in a vendor-specific namespace. It is recommended to separate vendor-specific types and vendor-specific instances into two or more namespaces.	Mandatory

In addition to the list of namespaces listed in O-PAS Part 6.2, Table 6 provides a list of namespaces and their index used for BrowseNames in this document when referring to OPC UA nodes. The default namespace of this document is not listed since all BrowseNames without a prefix use this default namespace.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Table 6: Additional Namespaces Used in this Document

NamespaceURI	Namespace Index	Example
https://www.opengroup.org/open-process-automation/O-PAS/Base	4	4:FunctionBlockType
https://www.opengroup.org/open-process-automation/O-PAS/FunctionBlocks	5	5:AnalogInputBlockType

6.5 Mapping of IEC 61131-3 Data Types to UA Variables and Data Types

Each O-PAS IEC 61131-3 execution engine shall use the mapping between IEC 61131-3 data types and OPC UA data types defined in OPC 30000 created by PLCopen, §5.2 and §5.3.1 with the following exceptions [CRQ-P6.6-0034]:

- The IEC 61131-3 data types OPAS_<Type>_SIGNAL_TYPE defined in Appendix A are not represented as structured data types in the OPC UA

  They shall be mapped to a single UA variable with the corresponding variable and data type as shown in Table 9 [CRQ-P6.6-0035]. REF_TO shall be treated like the data type to which it is pointing [CRQ-P6.6-0112].

- When mapping to UA attributes (instead of complete UA variables) other than the Value member of the Value attribute, the type mapping in Table 7 shall be used [CRQ-P6.6-0113]

- For UA variables used as the EnumValues property (UA data type is an array of the structured type EnumValueType) of UA variables of type MultiStateValueDiscreteType (or sub-type thereof), the IEC 61131-3 data type used shall be ARRAY[] OF LINT [CRQ-P6.6-0114]

  The values defined for the EnumValues property in the nodeset XML file describing the UA variable type to which the property belongs shall be used as the list of possible values that can be used in the array in the OPC UA server [CRQ-P6.6-0115]. Out of this list of possible values, only those whose numeric values are in the IEC 61131-3 ARRAY shall be presented to the user in the OPC UA server [CRQ-P6.6-0116]. The value 9223372036854775807 (= Int64 max) shall be used to indicate a temporary unused element in the IEC 61131-3 ARRAY [CRQ-P6.6-0117]. If a client tries to write to the MultiStateDiscreteType variable, a numeric value which is currently not part of the EnumValues property that write shall return the error Bad_OutOfRange [CRQ-P6.6-0131] and the associated IEC 61131-3 variable shall not change its value [CRQ-P6.6-0132].

Table 7: IEC 61131 Data Types Mapping for UA Attributes

UA Attribute		IEC 61131-3 Data Type	Reference
Value	DataValue struct members		

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

UA Attribute		IEC 61131-3 Data Type	Reference
	statusCode	OPAS_SIGNAL_STATUS_TYPE	See Section A.7.
	sourceTimestamp	LDATE_AND_TIME	See IEC 61131-3.
AccessLevel		OPAS_ACCESS_LEVEL_TYPE	See Section A.16.
AccessLevelEx		OPAS_ACCESS_LEVEL_TYPE	See Section A.16.

Unless the attributes are explicitly mapped on their own, variables that are not using the O-PAS structs shall not provide a source timestamp [CRQ-P6.6-0118] and shall use Good Severity on the Status with no Substatus, or, in case of an issue in the execution engine and/or UA server, Bad/Uncertain Severity and suitable Substatus [CRQ-P6.6-0119]. The InfoBits shall be DataItem with no limit bits set [CRQ-P6.6-0036].

The AccessLevel of all variables depends on whether the variable is one using the OPAS_<Type>_SIGNAL_TYPE defined in Appendix A or not, any mappings for the AccessLevel attribute, and the value of the XML attribute "permittedAccessLevel" of the UAObjectVarliableLink XML element (see Section 4.3.2).

All UA writes to IEC 61131 variables shall be applied at the beginning to the execution cycle of the task in which the variables are being used [CRQ-P6.6-0044].

6.5.1 Non O-PAS Struct Variables

If the variable is not an O-PAS struct, then the AccessLevel is by design restricted to allow writes only to the value of the variable, not writing to the history, the status, or the timestamp. Further instance-specific restrictions are provided by the permittedAccessLevel XML attribute for this variable in the configuration, or if mapped, an attribute mapping to the AccessLevel or AccessLevelEx attributes.

The effective AccessLevel/AccessLevelEx for the OPC UA variable for this variable shall be the overlap of the design-restricted AccessLevel, the mapped attribute, and the permittedAccessLevel defined for the variable in the variable mapping in the XML [CRQ-P6.6-0037]. The bitwise logic expression is as follows:

```
AccessLevel := NOT (HistoryWrite OR StatusWrite OR TimestampWrite) AND
mappedAttributeValue AND permittedAccessLevel
```

6.5.2 O-PAS Struct Variables

If the variable is an O-PAS struct, then the AccessLevel of the variable depends on the permittedAccessLevel XML attribute for this variable and the AccessLevel element of the MetaData in the O-PAS struct. The effective AccessLevel for the OPC UA variable for this variable shall be the overlap of the permittedAccessLevel in the XML and the AccessLevel in the MetaData in the O-PAS struct [CRQ-P6.6-0039]. The bitwise logic expression is as follows:

```
AccessLevel := permittedAccessLevel AND struct.MetaData.AccessLevel
```

Some of the OPC UA variables for O-PAS structs can have OPC UA properties that provide additional information about the variable such as the EURange and EngineeringUnits for AnalogUntiRangeType variables.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

The rules for the AccessLevel of these properties are as follows:

- If the properties are defined on 0:DataItemType (i.e., Definition and ValuePrecision) the AccessLevel shall always be CurrentRead only [CRQ-P6.6-0040]
- Otherwise, the AccessLevel/AccessLevelEx of all OPC UA properties depends on the permittedAccessLevelScope XML attribute for this variable and whether these attributes are mapped explicitly for the individual properties

  Depending on the value of the attribute, the AccessLevel/AccessLevelEx of the properties shall be as follows [CRQ-P6.6-0041]:

  — **VariableOnly**: if AccessLevel/AccessLevelEx are explicitly mapped for a property, then that mapping shall be used for the AccessLevel/AccessLevelEx [CRQ-P6.6-0120]

  All properties without such a mapping shall have an AccessLevel of CurrentRead only, or CurrentRead and HistoryRead only [CRQ-P6.6-0042]

  — **VariableAndProperties**: if AccessLevel/AccessLevelEx are explicitly mapped for a property, then that mapping shall be used for the AccessLevel/AccessLevelEx [CRQ-P6.6-0121]

  All properties of the OPC UA variable without such a mapping shall have the same AccessLevel/AccessLevelEx as the variable itself [CRQ-P6.6-0043]. In this second case the AccessLevel of the properties shall follow any changes made to the AccessLevel of the variable at runtime [CRQ-P6.6-0122].

6.6 OPC UA Conformance Units

This section defines the corresponding conformance units for the OPC UA Information Model for this document.

The conformance units are required by the profile in Section 7.5.2.

6.6.1 Conformance Units

Table 8 defines the server-based conformance units. These units are related to O-PAS Part 6.6 information models.

Table 8: O-PAS Information Model Conformance Units

Category	Title	Description
DCN61131		
Server	DCN 61131 Execution Engine	Support one or more instances of OPAS61131ExecutionEngineType.
Server	DCN 61131 Function Block Group	Support one or more instances of OPAS61131FunctionBlockGroupType per instance of OPAS61131ExecutionEngineType.
Server	DCN 61131 Application-specific Object Types	Provide application-specific object types as described in Section 6.2.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Category	Title	Description
Server	DCN 61131 Application-specific Objects	Provide application-specific objects as described in Section 6.3.
Server	DCN 61131 Alias Placement	Support adding elements to the alias list of the server based on the application currently executing (see OFCB_OUT in Section 7.2 and Section 6.3).
Server	DCN 61131 ConnectedToOutput References	Provide the 4:ConnectedToOutput references between variables in the different blocks to show their connection in the OPC UA server as described in Section 6.3.
Client	DCN 61131 Alias Subscription	Support resolving aliases using the Global Discovery Server (GDS) and subscribing to the values as required by the application currently loaded (see OFCB_IN in Section 7.2).

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

7 Function Blocks and IEC 61131 Extensions

This chapter deals with the IEC 61131-3 function blocks that an IEC 61131-3 runtime system must provide to its users in order to be conformant with this document. The function blocks allow application engineers to use the O-PAS interfaces.

7.1 Function Blocks Defined in O-PAS Part 6.4

An IEC 61131-3 O-PAS execution engine shall support the standard set of function blocks for IEC 61131 described in O-PAS Part 6.4 [CRQ-P6.6-0045].

7.2 O-PAS Connectivity Framework Blocks (OCFBs)

7.2.1 Overview

An O-PAS IEC 61131-3 application already provides its variables to the OCF/OPC UA by mapping variable declarations to OPC UA variables. However, this does not allow other DCNs to create subscriptions using aliases to any of those variables to transfer control data or specify a PubSub publishing for them. Further, it does not allow subscriptions to be created to UA variables on another DCN made available to the OCF by an alias pointing to them. Therefore, special function blocks called O-PAS Connectivity Framework Blocks (OCFBs) are defined in this section.

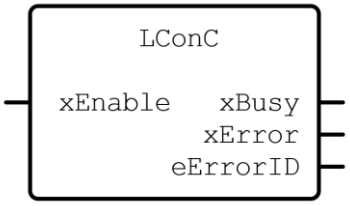


Figure 17: LConC Block as Defined by PLCopen

The OCFBs in this section all follow the LConC behavior model created by PLCopenu (see block illustration in Figure 17 and state model in Figure 18). One addition is made to the behavior in the form of the xAutoReset input: Usually there must be a falling edge on xEnable to cause the transition to the resetting state. When xAutoReset is TRUE, then the block shall automatically transition from error to resetting, even if there is no change of xEnable [CRQ-P6.6-0046]. Programmatically this modification means that xResetRequest is now (NOT xEnable) OR xAutoReset.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

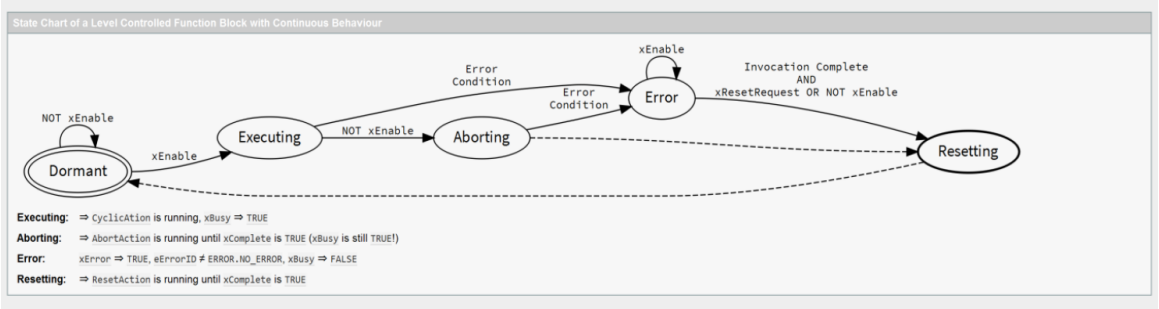


Figure 18: LConC Behavior Model Defined by PLCopen (from PLCopen Recommendation)

Since OCFBs operate in an OPC UA context, many of the possible errors can be connected to the OPC UA. Thus, a lot of the possible error codes in eErrorID relate to UA status. The eErrorID output of all OCFBs shall use the OCFB_ERROR data type defined in Section 7.2.1.1 [CRQ-P6.6-0047].

All OCFBs are FINAL and it shall not be possible to create sub-types from them [CRQ-P6.6-0048].

In order to accommodate all the different data types, a large set of OCFBs, each dedicated to one specific data type, was created. Table 9 lists all the OCFBs and the corresponding IEC 61131 types and OPC UA types.

Table 9: IEC 61131 and OPC UA Types Used for/by OCFBs

OCFB	IEC 61131 Simple Data Type of Primary Value	Structured Type for QST Plus Engineering Unit and Range	OPC UA Data Type	Required UA Variable Type (or Sub-Types Thereof)
OCFB_IN_REAL OCFB_OUT_REAL	REAL	OPAS_REAL_SIGNAL_TYPE	Float	0:AnalogUnitRangeType
OCFB_IN_LREAL OCFB_OUT_LREAL	LREAL	OPAS_LREAL_SIGNAL_TYPE	Double	0:AnalogUnitRangeType
OCFB_IN_LINT OCFB_OUT_LINT	LINT	OPAS_LINT_SIGNAL_TYPE	Int64	0:AnalogUnitRangeType or 0:MultiStateValueDiscreteType
OCFB_IN_ULINT OCFB_OUT_ULINT	ULINT	OPAS_ULINT_SIGANL_TYPE	UInt64	0:AnalogUnitRangeType or 0:MultiStateValueDiscreteType
OCFB_IN_BOOL OCFB_OUT_BOOL	BOOL	OPAS_BOOL_SIGNAL_TYPE	Boolean	0:TwoStateDiscreteItemType

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

OCFB	IEC 61131 Simple Data Type of Primary Value	Structured Type for QST Plus Engineering Unit and Range	OPC UA Data Type	Required UA Variable Type (or Sub-Types Thereof)
OCFB_TRIGGER_IN OCFB_TRIGGER_OUT	BOOL	N/A	Boolean	4:TriggerType

7.2.1.1 *Enum: OCFB_ERROR*

An OCFB_ERROR shall be represented as an IEC 61131-3 TYPE based on UDINT with the following specification [CRQ-P6.6-0111]. Table 10 shows the bit-pattern for the numerical values of the enumeration which integrates existing OPC UA StatusCodes into the enumeration:

Table 10: OCFB_Enum Numeric Value Bit-Pattern

Field	Bit Range	Description
Status	31:16	If IsOPCUA is 01: See Table 11 for symbolic names and numeric values. If IsOPCUA is 00: Top half (Severity + subcode) of OPC UA StatusCode as defined in OPC UA Part 4 and OPC UA Part 6. The OPC UA-specified SymbolName for the Substatus is used as the symbolic name for the enum value.
IsOPCUA	15:14	Determines content of ‘Status’: • 01 means error code as defined by this document in Table 11 • 00 means OPC UA StatusCode • 1x is reserved for future use
Reserved	13:0	Reserved for future use. Shall always be 0.

The OPC UA status for “Good” without Substatus (0x00000000) is used to indicate no error.

7.2.1.2 *OCFB_IN Behavior*

In the Executing state (see Figure 18), an OCFB_IN shall go through the following substates:

1. Look up alias in local server. If found go to Step 5.
2. Connect to the External Alias Server*.
3. Look up alias in the ExternalAliasServer.
4. Connect to the UA server with the variable to which the alias points*.
5. Verify that the UA variable and data type of the target are compatible with the OCFB used.
6. Create a UA subscription to the target of the alias.
7. Receive subscription updates.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

* Multiple OCFB_INs in the same application can share OPC UA connections for better efficiency.

The outputs of the OCFB_IN shall only update when the block is called in the application, not when new values arrive through the UA subscription [CRQ-P6.6-0049].

During the operation of the OCFB_IN, errors can arise. The nature of the error is indicated by setting the eErrorID output, and if the block has a signal output, the status in that signal. The errors in Table 11 are possible.

Table 11: OCFB_IN Specific Error Codes

Error	Symbolic Name and Numeric Value	Description
GDS Configuration Error	BadGDSConfigError (0x00014000)	The configuration for the GDS in this DCN is invalid; e.g., because the configured endpoint is not a valid endpoint.
GDS Connection Error	BadGDSConnectError (0x00024000)	It was not possible to connect to the GDS.
Alias Not Found	BadAliasNotFound (0x00034000)	The OCFB was unable to find the configured alias locally and in the GDS.
Target Connection Error	Applicable Bad UA Standard StatusCode	The OCFB could not connect to the UA server that contains the target of the alias.
Subscription Error	Applicable Bad UA Standard StatusCode	The OCFB could not create a subscription to the target of the alias or the subscription errored out.
UA Type Mismatch	BadOPASSignalMissmatch (0x00040000)	The UA variable type of the alias target is incompatible with the OCFB used. See Table 9.

If no error is present, eErrorID shall be set to an applicable Uncertain or Bad Substatus while the UA subscription is being established, or otherwise be set to the UA StatusCode for Good without a subcode [CRQ-P6.6-0050].

In the Dormant state of the block (see Figure 18) all subscriptions opened by this OCFB shall be closed [CRQ-P6.6-0051].

The aliases to which OCFB_IN connects are as configured in the AML configuration, not in the IEC 61131 code. Thus, there is no pin on the function block for the alias.

In AML OCFB_IN shall be represented by instances of the SUC Basic-OCFB_IN defined in O-PAS Part 6.2 [CRQ-P6.6-0052].

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

7.2.1.3 OCFB_OUT Behavior

OCFB_OUT is much simpler in its Executing state than OCFB_IN. All it shall do in this state is update the value of the UA variable every time the block is executed [CRQ-P6.6-0053].

Any error occurring internal to the OCFB_OUT shall cause the appropriate UA error code to be set on the UA variable and on the eErrorID output of the OCFB. If no error is present, it shall be set to the UA StatusCode for Good without a subcode [CRQ-P6.6-0054].

In the Dormant state OCFB_OUTs shall stop updating the UA variable in the server and the status of said variable shall be set to BadOutOfService [CRQ-P6.6-0055].

The aliases under which OCFB_OUT provides its variables are as configured in the AML configuration, not in the IEC 61131 code. Thus, there is no pin on the function block for the alias.

In AML OCFB_OUT shall be represented by instances of the SUC Basic-OCFB_OUT defined in O-PAS Part 6.2 [CRQ-P6.6-0056].

7.2.2 Function Block: OCFB_Trigger_IN

This function block, illustrated in Figure 19, allows O-PAS IEC 61131-3 applications to subscribe to a trigger on the OCF using its alias name and make it available in the IEC 61131-3 application.

In substate 7 of the Executing state the block shall behave as follows: a trigger arriving in the UA server shall be forwarded to the IEC space as a rising of the trigger VAR_OUTPUT. Trigger shall go to high/true on the first call of the function block after the trigger arrived via the OPC UA and stay high until the next call. If two triggers arrive simultaneously, they are both translated into consecutive rising edges with falling edges in between [CRQ-P6.6-0057].

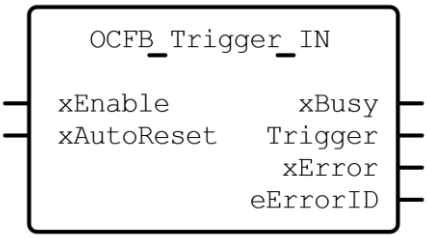


Figure 19: OPAS OCFB_Trigger_IN Function Block

In the OPC UA server OCFB_Trigger_IN shall be represented as instances of the OCFB_InType UA object type defined in O-PAS Part 6.2. The variable type of the ReceivedValue variable shall be TriggerType, also defined in O-PAS Part 6.2 [CRQ-P6.6-0058].

FB-Name	**OCFB_Trigger_IN**	
This function block is used to receive TriggerInputs as defined in O-PAS Part 6.2.		
VAR_INPUT		
xEnable	BOOL	See LConC.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

FB-Name	OCFB_Trigger_IN	
xAutoReset	BOOL	If TRUE, then xResetAction is always TRUE independent of xEnable. This means the block resets after an error even if there are no edges on xEnable.
VAR_OUTPUT		
xBusy	BOOL	See LConC.
Trigger	BOOL	Trigger has a rising edge for each trigger that arrives via the OPC UA. Each rising edge lasts at least from one function block execution to the next execution.
xError	BOOL	See LConC.
eErrorID	OCFB_ERROR	See Sections 7.2.1.1 and 7.2.1.2.

7.2.3 Function Block: OCFB_Trigger_OUT

This function block, illustrated in Figure 20, allows IEC 61131-3 applications to provide triggers to the OCF.

In substate 7 of the Executing state the block shall behave as follows: when there is a rising edge on the trigger VAR_INPUT of the function block when the function block is executed the OPC UA server shall send out the trigger [CRQ-P6.6-0059].

Figure 20: OPAS OCFB_Trigger_OUT Function Block

In the OPC UA server OCFB_Trigger_OUT shall be represented as instances of the OCFB_OUT UA object type defined in O-PAS Part 6.2 that utilizes the TriggerType UA variable type, also defined in O-PAS Part 6.2, for their OutgoingVariable variable [CRQ-P6.6-0060].

FB-Name	OCFB_Trigger_OUT	
This function block is used to receive TriggerOutputs as defined in O-PAS Part 6.2.		
VAR_INPUT		
Trigger	BOOL	A rising edge tells the function block to send out the trigger via the OPC UA.
xEnable	BOOL	See LConC.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

FB-Name	OCFB_Trigger_OUT	
xAutoReset	BOOL	If TRUE, then xResetAction is always TRUE independent of xEnable. This means the block resets after an error even if there are no edges on xEnable.
VAR_OUTPUT		
xBusy	BOOL	See LConC.
xError	BOOL	See LConC.
eErrorID	OCFB_ERROR	See Sections 7.2.1.1 and 7.2.1.2.

7.2.4 Function Block: OCFB_IN_<Type>

This function block, illustrated in Figure 21, makes it possible to subscribe to a signal on the OCF using its alias name and make it available in the IEC 61131-3 application as a variable. OCFB_IN exists for multiple data types; however, the behavior for all of them is the same. This section uses a generic definition using a <Type> placeholder. All possible replacements for <Type> are listed in Table 9.

In substate 7 of the Executing state the block shall behave as follows: on execution, the most recently received value from the OPC UA subscription shall be used for the value, and VSTEU VAR_OUTPUTs [CRQ-P6.6-0061].

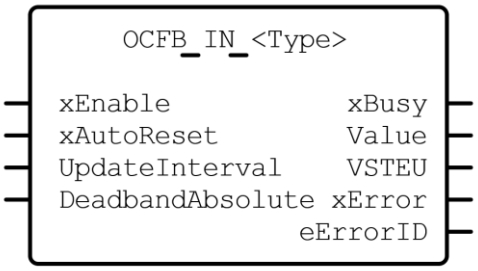


Figure 21: OPAS OCFB_IN_<Type> Function Block

In the OPC UA server OCFB_IN_<Type> shall be represented as instances of the OCFB_InType UA object type defined in O-PAS Part 6.2 that utilize the UA variable type defined in Table 9 for their OutgoingVariable variable [CRQ-P6.6-0062].

FB-Name	OCFB_IN_<Type>	
This function block is used for linkable inputs to a function block as defined in O-PAS Part 6.2.		
VAR_INPUT		
xEnable	BOOL	See LConC.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

FB-Name	OCFB_IN_<Type>	
xAutoReset	BOOL	If TRUE, then xResetAction is always TRUE independent of xEnable. This means the block resets after an error even if there are no edges on xEnable.
UpdateInterval	LTIME	The frequency in which the value should be updated via the OPC UA. The value VAR_OUTPUT only updates when the function block is executed though.
DeadbandAbsolute	LREAL	The Absolute Deadband to be used for OPC UA updates of this variable. See DataChangeFilter in OPC UA Part 4.
VAR_OUTPUT		
xBusy	BOOL	See LConC.
Value	<Type>	The current value of the input as received via the OPC UA.
VSTEU	OPAS_<Type>_SIGNAL_TYPE	The current value of the input as received via the OPC UA including Quality/Status, Timestamp, and Engineering Units and Range.
xError	BOOL	See LConC.
VAR		
Default	OPAS_<Type>_SIGNAL_TYPE	On the very first call of the function block the value output is set to this value. The default can only be set by initializing it.

7.2.5 Function Block: OCFB_OUT_<Type>

This function block, illustrated in Figure 22, allows IEC 61131-3 applications to provide signals with an alias to the OCF.

In substate 7 of the Executing state the block shall behave as follows: the value of the OPC UA variable is updated to Value [CRQ-P6.6-0063].

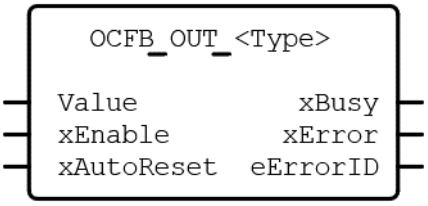


Figure 22: OPAS OCFB_OUT_<Type> Function Block

In the OPC UA server OCFB_OUT_<Type> shall be represented as instances of the OCFB_OutType UA object type defined in O-PAS Part 6.2 that utilize the UA variable type defined in Table 9 for their OutgoingVariable variable [CRQ-P6.6-0064].

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

FB-Name	OCFB_OUT_<Type>	
This function block is used for block outputs to a function block as defined in O-PAS Part 6.2.		
VAR_INPUT		
Value	OPAS_<Type>_SIGNAL_STRUCT	The value, timestamps, and status code to be used for the UA variable in the OPC UA server. Status code might be worsened by the server.
xExecute	BOOL	See LConC.
xAutoReset	BOOL	If TRUE, then xResetAction is always TRUE independent of xEnable. This means the block resets after an error even if there are no edges on xEnable.
VAR_OUTPUT		
xBusy	BOOL	See LConC.
xError	BOOL	See LConC.
eErrorID	OCFB_ERROR	See Sections 7.2.1.1 and 7.2.1.2.

7.3 Units of Measure

In the IEC 61131-3 application engineering units are expressed using the struct OPAS_EU_INFO_TYPE which stores engineering units efficiently as a combination of units of measure UnitId and NamespaceId, both of which are integers.

The unit of measure NamespaceId is a unique Id assigned to a unit of measure NamespaceUri. This Id to URI mapping is only valid within the application. Within the application translation between NamespaceUri and NamespaceId is enabled with the UOM_NS_ID() function.

In OPC UA engineering units consist of NamespaceUri, UnitId, Description, and Display Name. When a unit of measure stored in an OPAS_EU_INFO_TYPE in the IEC 61131 application is shared on the OCF, the OPC UA server and the firmware can take the two Ids stored in the struct and translate them into full OPC UA engineering units.

Note that the unit of measure Namespace is *not* related to the OPC UA namespace.

7.3.1 Function: UOM_NS_ID()

The function UOM_NS_ID() transforms a (unit of measure) NamespaceUri to a (unit of measure) NamespaceId that is used in the OPAS_EU_INFO_TYPE struct (see Section A.10) that is used to express engineering units within IEC 61131-3 applications. The assignment of NamespaceUri to NamespaceId shall be one-to-one and the UNECE URI of “http://www.opcfoundation.org/UA/units/un/cefact” shall always be NamespaceId 0 [CRQ-P6.6-0065].

```
FUNCTION UOM_NS_ID : DINT
VAR_INPUT
    NamespaceUri : STRING(255);
```


© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

```
END_VAR
END_FUNCTION
```

If the VAR_INPUT NamespaceUri is provided by a constant, the function call can be replaced with the correct NamespaceId for this NamespaceUri at compile time, if said NamespaceId is known at compile time.

7.4 IEC 61131-3 NAMESPACEs

Note: This section talks about IEC 61131-3 NAMESPACE, and not OPC UA Namespaces.

The O-PAS Standard uses a top-level IEC 61131-3 NAMESPACE named “org.opengroup.opas”. All IEC 61131-3 type definitions created in the O-PAS Standard are in this namespace either directly or in a nested IEC 61131-3 namespace. Only types and nested namespaces defined in the O-PAS Standard shall be allowed inside the “org.opengroup.opas” NAMESPACE [CRQ-P6.6-0066].

OCFBs (see Section 7.2), the data types used by them (Appendix A), and the Helper functions (Appendix B) shall use the IEC 61131-3 NAMESPACE “org.opengroup.opas.OCF” which is nested in the top-level “org.opengroup.opas” namespace [CRQ-P6.6-0067].

7.5 Function Block Conformance Units and Profiles

7.5.1 Conformance Units

Table 12 defines IEC 61131-3-based conformance units related to this document.

Table 12: Function Block Conformance Units

Title	Description
DCN61131-FBs	
DCN 61131 Base	Support IEC 61131-3.
DCN 61131 Standard Function Blocks	Support the O-PAS Part 6.4 function blocks (Section 7.1).
DCN 61131 OCFBs	Support the OCFBs: OCFB_Trigger_IN, OCFB_Trigger_OUT, OCFB_<Type>_IN, and OCFB_<Type>_OUT (Section 7.2).
DCN 61131 Data Types	Support the data types (Appendix A).
DCN 61131 Helper Functions	Support the Helper functions (Appendix B).
DCN 61131 Units of Measure Operator	Support the Units of Measure operator (Section 7.3).

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

7.5.2 Profiles

Table 13 defines the CTL-602 profile for the OPC UA server functionality necessary for a component that conforms to the O-PAS IEC 61131 capability defined in this document.

Table 13: O-PAS 61131 Function Blocks Profile Definition

Conformance Requirement	Conformance Unit/Facet/Profile Title	Defined in	Optional/ Mandatory
[CRQ-P6.6-0068]	OBC-203: O-PAS Function Block Compute Server Profile	O-PAS Part 6.2	M
[CRQ-P6.6-0069]	OCF-F-003: OCF Client Facet	O-PAS Part 4	M
[CRQ-P6.6-0123]	SEC-F-001: Security Facet	O-PAS Part 2	M
[CRQ-P6.6-0070]	OFB-001 61131 Profile	O-PAS Part 6.4	M
[CRQ-P6.6-0071]	DCN 61131 Base	Section 7.5.1	M
[CRQ-P6.6-0072]	DCN 61131 Standard Function Blocks	Section 7.5.1	M
[CRQ-P6.6-0073]	DCN 61131 OCFBs	Section 7.5.1	M
[CRQ-P6.6-0074]	DCN 61131 Data Types	Section 7.5.1	M
[CRQ-P6.6-0075]	DCN 61131 Helper Functions	Section 7.5.1	M
[CRQ-P6.6-0076]	DCN 61131 Units of Measure Function	Section 7.5.1	M
[CRQ-P6.6-0077]	DCN 61131 Execution Engine	Section 6.6.1	M
[CRQ-P6.6-0078]	DCN 61131 Function Block Group	Section 6.6.1	M
[CRQ-P6.6-0079]	DCN 61131 Application-specific Object Types	Section 6.6.1	M
[CRQ-P6.6-0080]	DCN 61131 Application-specific Objects	Section 6.6.1	M
[CRQ-P6.6-0081]	DCN 61131 Alias Placement	Section 6.6.1	M
[CRQ-P6.6-0082]	DCN 61131 ConnectedToOutput References	Section 6.6.1	O
[CRQ-P6.6-0083]	DCN 61131 Alias Subscription	Section 6.6.1	M

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

A Data Types (Normative)

This appendix defines data types that are used in O-PAS IEC 61131 applications when exchanging data with other DCNs via the OCF.

A.1 OPAS_BOOL_SIGNAL_TYPE

An OPAS_BOOL_SIGNAL_TYPE shall be represented as an IEC 61131-3 TYPE with the following specification [CRQ-P6.6-0084]. The structure elements are defined in Table 14.

```
TYPE OPAS_BOOL_SIGNAL_TYPE:
  STRUCT
    MetaData      : OPAS_SIGNAL_METADATA_TYPE ;
    Value         : BOOL ;
  END_STRUCT
END_TYPE
```

Table 14: OPAS_BOOL_SIGNAL_TYPE Elements

Structure Element	Data Type	Description
MetaData	OPAS_SIGNAL_METADATA_TYPE	OPC UA metadata for the signal such as Status, Source Timestamp, and AccessLevel.
Value	BOOL	The value of the signal represented as a Boolean.

A.2 OPAS_LINT_SIGNAL_TYPE

An OPAS_LINT_SIGNAL_TYPE shall be represented as an IEC 61131-3 TYPE with the following specification [CRQ-P6.6-0085]. The structure members are defined in Table 15.

```
TYPE OPAS_LINT_SIGNAL_TYPE:
  STRUCT
    MetaData      : OPAS_SIGNAL_METADATA_TYPE ;
    EURange       : OPAS_RANGE_LREAL_TYPE;
    EUInformation : OPAS_EU_INFO_TYPE;
    Value         : LINT ;
  END_STRUCT
END_TYPE
```


© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Table 15: OPAS_LINT_SIGNAL_TYPE Elements

Structure Element	Data Type	Description
MetaData	OPAS_SIGNAL_METADATA_TYPE	OPC UA metadata for the signal such as Status, Source Timestamp, and AccessLevel.
EURange	OPAS_RANGE_LREAL_TYPE	The range of the signal value in engineering units.
EUInformation	OPAS_EU_INFO_TYPE	The engineering unit information of the signal.
Value	LINT	The value of the signal represented as a 64-bit signed integer.

A.3 OPAS_ULINT_SIGNAL_TYPE

An OPAS_ULINT_SIGNAL_TYPE shall be represented as an IEC 61131-3 TYPE with the following specification [CRQ-P6.6-0086]. The structure members are defined in Table 16.

```
TYPE OPAS_ULINT_SIGNAL_TYPE:
  STRUCT
    MetaData      : OPAS_SIGNAL_METADATA_TYPE ;
    EURange       : OPAS_RANGE_LREAL_TYPE;
    EUInformation : OPAS_EU_INFO_TYPE;
    Value         : ULINT ;
  END_STRUCT
END_TYPE
```

Table 16: OPAS_ULINT_SIGNAL_TYPE Elements

Structure Element	Data Type	Description
MetaData	OPAS_SIGNAL_METADATA_TYPE	OPC UA metadata for the signal such as Status, Source Timestamp, and AccessLevel.
EURange	OPAS_RANGE_LREAL_TYPE	The range of the signal value in engineering units.
EUInformation	OPAS_EU_INFO_TYPE	The engineering unit information of the signal.
Value	ULINT	The value of the signal represented as a 64-bit unsigned integer.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

A.4 OPAS_REAL_SIGNAL_TYPE

An OPAS_REAL_SIGNAL_TYPE shall be represented as an IEC 61131-3 TYPE with the following specification [CRQ-P6.6-0087]. The structure members are defined in Table 17.

```
TYPE OPAS_REAL_SIGNAL_TYPE:
  STRUCT
    MetaData      : OPAS_SIGNAL_METADATA_TYPE ;
    EURange       : OPAS_RANGE_REAL_TYPE ;
    EUInformation : OPAS_EU_INFO_TYPE;
    Value         : REAL ;
  END_STRUCT
END_TYPE
```

Table 17: OPAS_REAL_SIGNAL_TYPE Elements

Structure Element	Data Type	Description
MetaData	OPAS_SIGNAL_METADATA_TYPE	OPC UA metadata for the signal such as Status, Source Timestamp, and AccessLevel.
EURange	OPAS_RANGE_REAL_TYPE	The range of the signal value in engineering units.
EUInformation	OPAS_EU_INFO_TYPE	The engineering unit information of the signal.
Value	REAL	The value of the signal represented as a 32-bit floating point number.

A.5 OPAS_LREAL_SIGNAL_TYPE

An OPAS_LREAL_SIGNAL_TYPE shall be represented as an IEC 61131-3 TYPE with the following specification [CRQ-P6.6-0088]. The structure members are defined in Table 18.

```
TYPE OPAS_LREAL_SIGNAL_TYPE:
  STRUCT
    MetaData      : OPAS_SIGNAL_METADATA_TYPE ;
    EURange       : OPAS_RANGE_LREAL_TYPE ;
    EngineeringUnits : OPAS_EU_INFO_TYPE;
    Value         : LREAL ;
  END_STRUCT
END_TYPE
```

Table 18 - OPAS_LREAL_SIGNAL_TYPE Elements

Structure Element	Data Type	Description
MetaData	OPAS_SIGNAL_METADATA_TYPE	OPC UA metadata for the signal such as Status, Source Timestamp, and AccessLevel.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Structure Element	Data Type	Description
EURange	OPAS_RANGE_LREAL_TYPE	The range of the signal value in engineering units.
EngineeringUnits	OPAS_EU_INFO_TYPE	The engineering unit information of the signal.
Value	LREAL	The value of the signal represented as a 64-bit floating point number.

A.6 OPAS_SIGNAL_METADATA_TYPE

An OPAS_SIGNAL_METADATA_TYPE shall be represented as an IEC 61131-3 TYPE with the following specification [CRQ-P6.6-0089]. The structure members are defined in Table 19.

Note: The OPAS_SIGNAL_METADATA_TYPE is the IEC 61131-3 representation of the extra metadata provided with an OPC UA "DataValue" defined in OPC UA Part 4.

```
TYPE OPAS_SIGNAL_METADATA_TYPE :
  STRUCT
    Status          : OPAS_SIGNAL_STATUS_TYPE ;
    AccessLevel     : OPAS_ACCESS_LEVEL_TYPE ;
    SourceTimestamp : LDATE_AND_TIME ;
  END_STRUCT
END_TYPE
```

Table 19: OPAS_SIGNAL_METADATA_TYPE Elements

Structure Element	Description
Status	Contains the quality status of the value.
AccessLevel	Contains the OPC UA access level for the variable.
SourceTimeStamp	Contains the timestamp that was applied to the value by the data source.

A.7 OPAS_SIGNAL_STATUS_TYPE

An OPAS_SIGNAL_STATUS_TYPE shall be represented as an IEC 61131-3 TYPE that is an alias type (directly derived data type) for UDINT [CRQ-P6.6-0090].

Helper functions to access the data stored in this type are defined in Appendix B.

Note: The OPAS_SIGNAL_STATUS_TYPE is the IEC 61131-3 representation of the OPC UA "StatusCode" type defined in OPC UA Part 4.

```
TYPE
  OPAS_SIGNAL_STATUS_TYPE : UDINT;

END_TYPE
```


© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

A.8 OPAS_RANGE_REAL_TYPE

An OPAS_RANGE_REAL_TYPE shall be represented as an IEC 61131-3 TYPE with the following specification [CRQ-P6.6-0091]. The structure members are defined in Table 20.

Note: The OPAS_RANGE_REAL_TYPE is the IEC 61131-3 representation of the OPC UA "Range" data type defined in OPC UA Part 8. In OPC UA the range is represented by double-precision floating point numbers and in this IEC 61131 type single-precision is used. This means that conversions are needed in the firmware to share an OPAS_RANGE_REAL_TYPE via the OCF/OPC UA.

```
TYPE OPAS_RANGE_REAL_TYPE:
  STRUCT
    Low  : REAL ;
    High : REAL ;
  END_STRUCT
END_TYPE
```

Table 20: OPAS_RANGE_REAL_TYPE Elements

Structure Element	Description
Low	The low value of the range represented as a 32-bit floating point number.
High	The high value of the range represented as a 32-bit floating point number.

A.9 OPAS_RANGE_LREAL_TYPE

An OPAS_RANGE_LREAL_TYPE shall be represented as an IEC 61131-3 TYPE with the following specification [CRQ-P6.6-0092]. The structure members are defined in Table 21.

Note: The OPAS_RANGE_LREAL_TYPE is the IEC 61131-3 representation of the OPC UA "Range" data type defined in OPC UA Part 8.

```
TYPE OPAS_RANGE_LREAL_TYPE:
  STRUCT
    Low  : LREAL ;
    High : LREAL ;
  END_STRUCT
END_TYPE
```

Table 21: OPAS_RANGE_LREAL_TYPE Elements

Structure Element	Description
Low	The low value of the range represented as a 64-bit floating point number.
High	The high value of the range represented as a 64-bit floating point number.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

A.10 OPAS_EU_INFO_TYPE

An OPAS_EU_INFO_TYPE shall be represented as an IEC 61131-3 TYPE with the following specification [CRQ-P6.6-0093]. The structure elements are defined in Table 22.

Note: The OPAS_EU_INFO_TYPE is an indirect IEC 61131-3 representation of the OPC UA "EUInformation" data type defined in OPC UA Part 8. The NamespaceID and UnitID can be used to extract all the information needed to form EUInformation from the AML file.

```
TYPE OPAS_EU_INFO_TYPE:
  STRUCT
    NamespaceID : DINT;
    UnitID      : DINT;
  END_STRUCT
END_TYPE
```

Table 22: OPAS_EU_INFO_TYPE Elements

Structure Element	Description
NamespaceID	Identifies the organization (company, standards organization) that defines the engineering unit information. Needs to be looked up using the NamespaceUri with a function call (see Section 7.3.1).
UnitID	Identifier for programmatic evaluation. −1 is used if an Id number representing the unit is not available.

A.11 OPAS_SIGNAL_SEVERITY_TYPE

An OPAS_SIGNAL_SEVERITY_TYPE shall be represented as an IEC 61131-3 TYPE based on UDINT with the following specification [CRQ-P6.6-0094]. The enumeration elements are defined in Table 23.

Note: The OPAS_SIGNAL_SEVERITY_TYPE is the IEC 61131-3 representation of the OPC UA "Severity" field of the StatusCode defined in OPC UA Part 4.

```
TYPE OPAS_SIGNAL_SEVERITY_TYPE :
(
  Good      := 16#0000_0000,
  Uncertain := 16#4000_0000,
  Bad       := 16#8000_0000,
  Reserved  := 16#C000_0000
) UDINT;
END_TYPE
```

Table 23: OPAS_SIGNAL_SEVERITY_TYPE Elements

Enum Element	Description
Good	Indicates that the operation was successful and the associated results may be used.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Enum Element	Description
Uncertain	Indicates that the operation was partially successful and that associated results might not be suitable for some purposes.
Bad	Indicates that the operation to acquire the data failed and any associated results cannot be used.
Reserved	Reserved for future use. All function block code should treat this severity as “Bad”.

A.12 OPAS_SIGNAL_STATUS_SUBCODE_TYPE

An OPAS_SIGNAL_STATUS_SUBCODE_TYPE shall be represented as an IEC 61131-3 TYPE with the following specification [CRQ-P6.6-0095]. The meaning of each enumeration is defined in OPC UA Part 4 and the mapping to numeric values is provided by OPC UA Part 6, Annex A.2.

Note: The OPAS_SIGNAL_STATUS_SUBCODE_TYPE is the IEC 61131-3 representation of the OPC UA “SubCode” field of the StatusCode defined in OPC UA Part 4.

```
TYPE OPAS_SIGNAL_STATUS_SUBCODE_TYPE:
(
  // for Enum Values see CSV file linked in Annex a.2 of OPC UA Part 6
) UDINT;
END_TYPE
```

A.13 OPAS_SIGNAL_INFO_TYPE

An OPAS_SIGNAL_INFO_TYPE shall be represented as an IEC 61131-3 TYPE based on UDINT with the following specification [CRQ-P6.6-0096]. The enum elements are defined in Table 24.

Note: The OPAS_SIGNAL_INFO_TYPE is the IEC 61131-3 representation of the OPC UA “InfoType” field of the StatusCode defined in OPC UA Part 4.

```
TYPE OPAS_SIGNAL_INFO_TYPE :
(
  NotUsed   := 16#0000_0000,
  DataValue := 16#0000_0100
) UDINT;
END_TYPE
```

Table 24: OPAS_SIGNAL_INFO_TYPE Elements

Enum Element	Description
NotUsed	The InfoBits are not used and are set to zero.
DataValue	The StatusCode and its InfoBits are associated with a data value.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

A.14 OPAS_SIGNAL_INFOBITS_TYPE

An OPAS_SIGNAL_INFOBITS_TYPE shall be represented as an IEC 61131-3 TYPE with the following specification [CRQ-P6.6-0097]. The structure elements are defined in Table 25.

Note: The OPAS_SIGNAL_INFOBITS_TYPE is the IEC 61131-3 representation of the OPC UA "InfoBits" field of the StatusCode defined in OPC UA Part 4.

```
TYPE OPAS_SIGNAL_INFOBITS_TYPE:
  STRUCT
    //HistorianBits : STRUCT; // Defined in OPC UA, not in the O-PAS Standard.
    Overflow : BOOL;
    LimitBits : OPAS_SIGNAL_LIMITBITS_TYPE;
  END_STRUCT
END_TYPE
```

Table 25: OPAS_SIGNAL_INFOBITS_TYPE Elements

Structure Element	Description
Overflow	If this bit is set, not every detected change has been returned since the data source's queue buffer for the signal reached its limit and had to purge data.
LimitBits	Identifies the limits associated with the data value.

A.15 OPAS_SIGNAL_LIMITBITS_TYPE

An OPAS_SIGNAL_LIMITBITS_TYPE shall be represented as an IEC 61131-3 TYPE based on UDINT with the following specification [CRQ-P6.6-0098]. The values are defined in Table 26.

Note: The OPAS_SIGNAL_LIMITBITS_TYPE is the IEC 61131-3 representation of the OPC UA "LimitBits" field of the StatusCode defined in OPC UA Part 4.

```
TYPE OPAS_SIGNAL_LIMITBITS_TYPE:
(
  None  := 16#0000_0000,
  Low   := 16#0000_0100,
  High  := 16#0000_0200,
  Const := 16#0000_0300
) UDINT;
END_TYPE
```

Table 26: OPAS_SIGNAL_LIMITBITS_TYPE Elements

Value	Description
None	The data value is not a constant and is free to change.
Low	The value is at the lower limit of the data source.
High	The value is at the upper limit of the data source.
Const	The value is a constant and cannot change.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

A.16 OPAS_ACCESS_LEVEL_TYPE

An OPAS_ACCESS_LEVEL_TYPE shall be represented as an IEC 61131-3 TYPE with the following specification [CRQ-P6.6-0099]. The structure elements are defined in Table 27.

Note: The OPAS_ACCESS_LEVEL_TYPE is the IEC 61131-3 representation of the OPC UA AccessLevelExType as defined in OPC UA Part 3.

```
TYPE OPAS_ACCESS_LEVEL_TYPE:
  STRUCT
    CurrentRead : BIT;
    CurrentWrite : BIT;
    HistoryRead : BIT;
    HistoryWrite : BIT;
    SemanticChange : BIT;
    StatusWrite : BIT;
    TimestampWrite : BIT;
    NonatomicRead : BIT;
    NonatomicWrite : BIT;
    WriteFullArrayOnly : BIT;
  END_STRUCT
END_TYPE
```

Table 27: OPAS_ACCESS_LEVEL_TYPE Elements (Descriptions from OPC UA Part 3)

Structure Element	Description
CurrentRead	Indicates if the current value is readable. It also indicates if the current value of the variable is available (0 means not readable, 1 means readable).
CurrentWrite	Indicates if the current value is writable. It also indicates if the current value of the variable is available (0 means not writable, 1 means writable).
HistoryRead	Indicates if the current value is writable. It also indicates if the current value of the variable is available (0 means not writable, 1 means writable).
HistoryWrite	Indicates if the history of the value is writable. It also indicates if the history of the variable is available via the OPC UA server (0 means not writable, 1 means writable).
SemanticChange	This flag is set for properties that define semantic aspects of the parent node of the property and where the property value, and thus the semantic, may change during operation (0 means is not a semantic, 1 means is a semantic).
StatusWrite	Indicates if the current StatusCode of the value is writable (0 means only StatusCode Good is writable, 1 means any StatusCode is writable).
TimestampWrite	Indicates if the current SourceTimestamp is writable (0 means only null timestamps are writable, 1 means any timestamp value is writeable).
NonatomicRead	Indicates non-atomicity for read access (0 means that atomicity is assured).
NonatomicWrite	Indicates non-atomicity for write access (0 means that atomicity is assured).
WriteFullArrayOnly	Indicates if write of IndexRange is supported (0 means write of IndexRange is supported).

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

B Helper Functions (Normative)

In order to make use of OPAS_SIGNAL_STATUS_TYPE easier there is a set of standard Helper functions that enable easy access to parts of the OPC UA status.

B.1 OPAS_SIGNAL_STATUS_IS_GOOD()

This function can be called to check if the severity inside an OPAS_SIGNAL_STATUS_TYPE variable is Good. If bits 30:31 are 00, then TRUE is returned, else FALSE is returned. The inputs and outputs of the function are defined in Table 28 [CRQ-P6.6-0100].

```
FUNCTION OPAS_SIGNAL_STATUS_IS_GOOD : BOOL
VAR_INPUT
  Status : OPAS_SIGNAL_STATUS_TYPE;
END_VAR
END_FUNCTION
```

Table 28: Helper Function OPAS_SIGNAL_STATUS_IS_GOOD()

Scope	Variable Name	Type	Description
VAR_INPUT	Status	OPAS_SIGNAL_STATUS_TYPE	Status to be checked to see if it is Good.
VAR_OUTPUT	OPAS_SIGNAL_STATUS_IS_GOOD	BOOL	RETURN value: TRUE if Status is Good, else FALSE.

B.2 OPAS_SIGNAL_STATUS_IS_UNCERTAIN()

This function can be called to check if the severity inside an OPAS_SIGNAL_STATUS_TYPE variable is UNCERTAIN. If bits 30:31 are 01, then TRUE is returned, else FALSE is returned. The inputs and outputs of the function are defined in Table 29 [CRQ-P6.6-0101].

```
FUNCTION OPAS_SIGNAL_STATUS_IS_UNCERTAIN : BOOL
VAR_INPUT
  Status : OPAS_SIGNAL_STATUS_TYPE;
END_VAR
END_FUNCTION
```

Table 29: Helper Function OPAS_SIGNAL_STATUS_IS_UNCERTAIN()

Scope	Variable Name	Type	Description
VAR_INPUT	Status	OPAS_SIGNAL_STATUS_TYPE	Status to be checked to see if it is Uncertain.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Scope	Variable Name	Type	Description
VAR_OUTPUT	OPAS_SIGNAL_STATUS_IS_UNCERTAIN	BOOL	RETURN value: TRUE if Status is Uncertain, else FALSE.

B.3 OPAS_SIGNAL_STATUS_IS_BAD()

This function can be called to check if the severity inside an OPAS_SIGNAL_STATUS_TYPE variable is Bad. If bits 30:31 are 10, then TRUE is returned, else FALSE is returned. The inputs and outputs of the function are defined in Table 30 [CRQ-P6.6-0102].

```
FUNCTION OPAS_SIGNAL_STATUS_IS_BAD : BOOL
VAR_INPUT
  Status : OPAS_SIGNAL_STATUS_TYPE;
END_VAR
END_FUNCTION
```

Table 30: Helper Function OPAS_SIGNAL_STATUS_IS_BAD()

Scope	Variable Name	Type	Description
VAR_INPUT	Status	OPAS_SIGNAL_STATUS_TYPE	Status to be checked to see if it is Bad.
VAR_OUTPUT	OPAS_SIGNAL_STATUS_IS_BAD	BOOL	RETURN value: TRUE if Status is Bad, else FALSE.

B.4 OPAS_SIGNAL_STATUS_GET_SUBCODE()

This function can be called to extract the subcode (bits 16:27) inside an OPAS_SIGNAL_STATUS_TYPE variable and get it as an OPAS_SIGNAL_STATUS_SUBCODE_TYPE. The inputs and outputs of the function are defined in Table 31 [CRQ-P6.6-0103].

```
FUNCTION OPAS_SIGNAL_STATUS_GET_SUBCODE : OPAS_SIGNAL_STATUS_SUBCODE_TYPE
VAR_INPUT
  Status : OPAS_SIGNAL_STATUS_TYPE;
END_VAR
END_FUNCTION
```

Table 31: Helper Function OPAS_SIGNAL_STATUS_GET_SUBCODE()

Scope	Variable Name	Type	Description
VAR_INPUT	Status	OPAS_SIGNAL_STATUS_TYPE	Status in which to get SubCode.
VAR_OUTPUT	OPAS_SIGNAL_STATUS_GET_SUBCODE	OPAS_SIGNAL_STATUS_SUBCODE_TYPE	RETURN value: SubCode of Status.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

B.5 OPAS_SIGNAL_STATUS_SET_SUBCODE()

This function can be called to set the subcode and severity (bits 30:31 and 16:27) inside an OPAS_SIGNAL_STATUS_TYPE variable. The inputs and outputs of the function are defined in Table 32 [CRQ-P6.6-0104].

```
FUNCTION OPAS_SIGNAL_STATUS_SET_SUBCODE : OPAS_SIGNAL_STATUS_TYPE
VAR_INPUT
  Status_In : OPAS_SIGNAL_STATUS_TYPE;
  SubCode : OPAS_SIGNAL_STATUS_SUBCODE_TYPE;
END_VAR
END_FUNCTION
```

Table 32: Helper Function OPAS_SIGNAL_STATUS_SET_SUBCODE()

Scope	Variable Name	Type	Description
VAR_INPUT	Status_In	OPAS_SIGNAL_STATUS_TYPE	Status in which to set the SubCode.
VAR_INPUT	SubCode	OPAS_SIGNAL_STATUS_SUBCODE_TYPE	The SubCode in which to set Status.
VAR_OUTPUT	OPAS_SIGNAL_STATUS_SET_SUBCODE	OPAS_SIGNAL_STATUS_TYPE	RETURN value: Status_In with the substate (and thus severity) replaced by Substatus.

B.6 OPAS_SIGNAL_STATUS_IS_DATAITEM()

This function can be called to check if the InfoType inside an OPAS_SIGNAL_STATUS_TYPE variable is DataItem. If bits 10:11 are 01, then TRUE is returned, else FALSE is returned. The inputs and outputs of the function are defined in Table 33 [CRQ-P6.6-0105].

```
FUNCTION OPAS_SIGNAL_STATUS_IS_DATAITEM : BOOL
VAR_INPUT
  Status : OPAS_SIGNAL_STATUS_TYPE;
END_VAR
END_FUNCTION
```

Table 33: Helper Function OPAS_SIGNAL_STATUS_IS_DATAITEM()

Scope	Variable Name	Type	Description
VAR_INPUT	Status	OPAS_SIGNAL_STATUS_TYPE	Status in which to check InfoBits.
VAR_OUTPUT	OPAS_SIGNAL_STATUS_IS_DATAITEM	BOOL	RETURN value: TRUE if InfoBits are set for DataItem, else FALSE.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

B.7 OPAS_SIGNAL_STATUS_GET_LIMITS()

This function can be called to extract the LimitBits inside an OPAS_SIGNAL_STATUS_TYPE variable and get it as an OPAS_SIGNAL_LIMITBITS_TYPE. The inputs and outputs of the function are defined in Table 34 [CRQ-P6.6-0106].

If the LimitBits are not present because the InfoType (as indicated by the InfoBits) is not DataItem, Success is set to TRUE and OPAS_SIGNAL_LIMITBITS_TYPE.None is returned.

```
FUNCTION OPAS_SIGNAL_STATUS_GET_LIMITS : OPAS_SIGNAL_LIMITBITS_TYPE
VAR_INPUT
  Status : OPAS_SIGNAL_STATUS_TYPE;
END_VAR
VAR_OUTPUT
  Success : BOOL;
END_VAR
END_FUNCTION
```

Table 34: Helper Function OPAS_SIGNAL_STATUS_GET_LIMITS()

Scope	Variable Name	Type	Description
VAR_INPUT	Status	OPAS_SIGNAL_STATUS_TYPE	Status from which to extract the LimitBits.
VAR_OUTPUT	Success	BOOL	Indicates whether LimitBits was present in the status (i.e., InfoType is DataItem).
VAR_OUTPUT	OPAS_SIGNAL_STATUS_GET_LIMITS	OPAS_SIGNAL_LIMITBITS_TYPE	RETURN value: the LimitBits Status, or if InfoBits is not DataItem, OPAS_SIGNAL_LIMITBITS_TYPE.None.

B.8 OPAS_SIGNAL_STATUS_SET_LIMITS()

This function can be called to set the LimitBits inside an OPAS_SIGNAL_STATUS_TYPE variable. When called, it also sets the InfoBits to DataItem. The inputs and outputs of the function are defined in Table 35 [CRQ-P6.6-0107].

```
FUNCTION OPAS_SIGNAL_STATUS_SET_LIMITS : OPAS_SIGNAL_STATUS_TYPE
VAR_INPUT
  Status_In : OPAS_SIGNAL_STATUS_TYPE;
  Limits : OPAS_SIGNAL_LIMITBITS_TYPE;
END_VAR
END_FUNCTION
```

Table 35: Helper Function OPAS_SIGNAL_STATUS_SET_LIMITS()

Scope	Variable Name	Type	Description
VAR_INPUT	Status_In	OPAS_SIGNAL_STATUS_TYPE	Status in which to set Limits.

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Scope	Variable Name	Type	Description
VAR_INPUT	Limits	OPAS_SIGNAL_LIMITBITS_TYPE	The LimitBits to set in Status_In.
VAR_OUTPUT	OPAS_SIGNAL_STATUS_SET_LIMITS	OPAS_SIGNAL_STATUS_TYPE	RETURN value: Status_In with the LimitBits replaced by Limits and InfoType set to DataItem.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

C IEC 61131-10 AddData (Normative)

C.1 XML Schema Definition (xsd)

The AddData elements using IEC 61131-10 XML shall use the following schema [CRQ-P6.6-0108]:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
           xmlns:tns="http://www.opengroup.org/open-process-automation/O-
PAS/61131-10/schema"
           targetNamespace="http://www.opengroup.org/open-process-automation/O-
PAS/61131-10/schema"
           elementFormDefault="qualified">

  <xsd:simpleType name="RelationshipId">
    <xsd:restriction base="xsd:ID"/>
  </xsd:simpleType>
  <xsd:attribute name="nodesetXMLrelId" type="tns:RelationshipId"/>

  <xsd:simpleType name="NumericNodeIdValue">
    <xsd:restriction base="xsd:unsignedInt"/>
  </xsd:simpleType>
  <xsd:simpleType name="GuidNodeIdValue">
    <xsd:restriction base="xsd:string">
      <xsd:pattern value="[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-
fA-F]{4}-[0-9a-fA-F]{12}" />
    </xsd:restriction>
  </xsd:simpleType>
  <xsd:simpleType name="OpaqueNodeIdValue">
    <xsd:restriction base="xsd:base64Binary"/>
  </xsd:simpleType>
  <xsd:simpleType name="StringNodeIdValue">
    <xsd:restriction base="xsd:string"/>
  </xsd:simpleType>
  <xsd:simpleType name="AccessLevelEx">
    <xsd:restriction base="xsd:unsignedInt"/>
  </xsd:simpleType>
  <xsd:attribute name="permittedAccessLevel" type="tns:AccessLevelEx"/>
  <xsd:simpleType name = "AccessLevelScope">
    <xsd:restriction base = "xsd:string">
      <xsd:enumeration value = "VariableOnly"/>
      <!-- Only the Main Variable -->
      <xsd:enumeration value = "VariableAndProperties"/>
      <!-- The Variable plus all variables from it wiht HasProperty
relationships -->
    </xsd:restriction>
  </xsd:simpleType>
  <xsd:attribute name="permittedAccessLevelScope" type="tns:AccessLevelScope"/>
  <xsd:simpleType name="NamespaceURI">
    <xsd:restriction base="xsd:anyURI"/>
  </xsd:simpleType>

  <xsd:complexType name="NodeIdValue">
    <xsd:choice minOccurs="1" maxOccurs="1">
      <xsd:element name="String" type="tns:StringNodeIdValue"/>
```


© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

```
      <xsd:element name="Numeric" type="tns:NumericNodeIdValue"/>
      <xsd:element name="Guid" type="tns:GuidNodeIdValue"/>
      <xsd:element name="Opaque" type="tns:OpaqueNodeIdValue"/>
    </xsd:choice>
  </xsd:complexType>

  <xsd:element name="NamespaceURI" type="tns:NamespaceURI"/>

  <xsd:complexType name="NodeId">
    <xsd:sequence minOccurs="1" maxOccurs="1">
      <xsd:element ref="tns:NamespaceURI" minOccurs="1" maxOccurs="1"/>
      <xsd:element name="Value" type="tns:NodeIdValue" minOccurs="1"
maxOccurs="1"/>
    </xsd:sequence>
  </xsd:complexType>

  <xsd:element name="NodeId" type="tns:NodeId" nillable="true"/>

  <xsd:complexType name="UAObjectTypeLink">
    <xsd:sequence minOccurs="1" maxOccurs="1">
      <xsd:element name="NodeId" type="tns:NodeId" minOccurs="1"
maxOccurs="1"/>
    </xsd:sequence>
    <xsd:attribute ref="tns:nodesetXMLrelId"/>
  </xsd:complexType>
  <xsd:element name="UAObjectTypeLink" type="tns:UAObjectTypeLink"/>

  <xsd:complexType name="UABrowsePathSegment">
    <xsd:sequence minOccurs="1" maxOccurs="1">
      <xsd:element ref="tns:NamespaceURI" minOccurs="0" maxOccurs="1"/>
      <xsd:element name="BrowsePathSegmentName" type="xsd:string" minOccurs="1"
maxOccurs="1"/>
    </xsd:sequence>
  </xsd:complexType>

  <xsd:complexType name="UAObjectVariableLink">
    <xsd:sequence minOccurs="1" maxOccurs="1">
      <xsd:element name="BroswePathSegment" type="tns:UABrowsePathSegment"
minOccurs="1" maxOccurs="unbounded"/>
    </xsd:sequence>
  </xsd:complexType>
  <xsd:attribute ref="tns:permittedAccessLevel" use="optional" default="1"/>
  <xsd:attribute ref="tns:permittedAccessLevelScope" use="optional"
default="VariableOnly"/>
  <xsd:element name="UAObjectVariableLink" type="tns:UAObjectVariableLink"/>

   <xsd:simpleType name="UAAttribute" final="restriction" >
    <xsd:restriction base="xsd:string">
      <xsd:enumeration value="Value_value" />
      <xsd:enumeration value="Value_sourceTimestamp" />
      <xsd:enumeration value="Value_statusCode" />
      <xsd:enumeration value="AccessLevel" />
      <xsd:enumeration value="AccessLevelEx" />
    </xsd:restriction>
  </xsd:simpleType>
  <xsd:attribute name="targetAttribute" type="tns:UAAttribute"/>

  <xsd:complexType name="UAObjectVariableAttributeLink">
    <xsd:sequence minOccurs="1" maxOccurs="1">
      <xsd:element name="BroswePathSegment" type="tns:UABrowsePathSegment"
minOccurs="1" maxOccurs="unbounded"/>
    </xsd:sequence>
    <xsd:attribute ref="tns:targetAttribute" use="required"/>
```


© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

```
  </xsd:complexType>
  <xsd:element name="UAObjectVariableAttributeLink"
type="tns:UAObjectVariableAttributeLink"/>
</xsd:schema>
```

C.2 AddDataInfo

The following AddDataInfo element shall be added to the ContentHeader of every IEC 61131-10 XML file in a .opas package [CRQ-P6.6-0109]:

```
<contentHeader name="logic model">
  ...
    <addDataInfo>
      <info name="http://www.opengroup.org/open-process-automation/O-PAS/61131-
10/schema" vendor="O-PAS">
        <description>
          <xhtml:p>Schema to integrate into an O-PAS package.</xhtml:p>
        </description>
      </info>
    </addDataInfo>
</contentHeader>
```

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

D O-PAS AutomationML Definitions (Normative)

Formal, normative AutomationML definitions can be found in the AML file at http://open-process-automation.projects.opengroup.org/twg/part-6-information-models/part-6.6-iec-61131-3/6.6_files/.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

E O-PAS Part 6.6 OPC UA Mappings (Normative)

E.1 Namespace and Identifiers for the O-PAS Part 6.6 Information Model

This appendix defines the numeric identifiers for all of the numeric *NodeIds* defined in the O-PAS Standard. The identifiers are specified in a CSV file with the following syntax:

```
<SymbolName>, <Identifier>, <NodeClass>
```

where the *SymbolName* is either the *BrowseName* of a *Type* node or the *BrowsePath* for an *Instance* node that appears in the specification and the *Identifier* is the numeric value for the *NodeId*. Multiple *NodeIds* in the same CSV file are separated by newline characters.

The *BrowsePath* for an *Instance* node is constructed by appending the *BrowseName* of the *Instance* node to the *BrowseName* for the containing instance or type. An underscore character is used to separate each *BrowseName* in the path.

The *NamespaceUri* for all *NodeIds* defined here is: https://www.opengroup.org/open-process-automation/O-PAS/61131.

The CSV of O-PAS numeric *NodeId* values released with the O-PAS Standard can be found here:

http://open-process-automation.projects.opengroup.org/twg/part-6-information-models/part-6.6-iec-61131-3/6.6_files/

A computer-processable version of the complete information model defined in the O-PAS Standard is also provided. It follows the XML information model schema syntax defined in the OPC UA Part 6.

The information model schema released with the O-PAS Standard can be found here:

http://open-process-automation.projects.opengroup.org/twg/part-6-information-models/part-6.6-iec-61131-3/6.6_files/

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

E.2 Profile URIs for the O-PAS Information Model

Table 36 defines the profile URIs for the profiles defined in O-PAS Part 6.6. All implementations must use the defined URIs to uniquely identify the corresponding profile.

Table 36: Profile URIs

Profile	Profile URI
CTL-602 O-PAS 61131 Function Blocks Profile	https://www.opengroup.org/open-process-automation/O-PAS/CTL-602

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

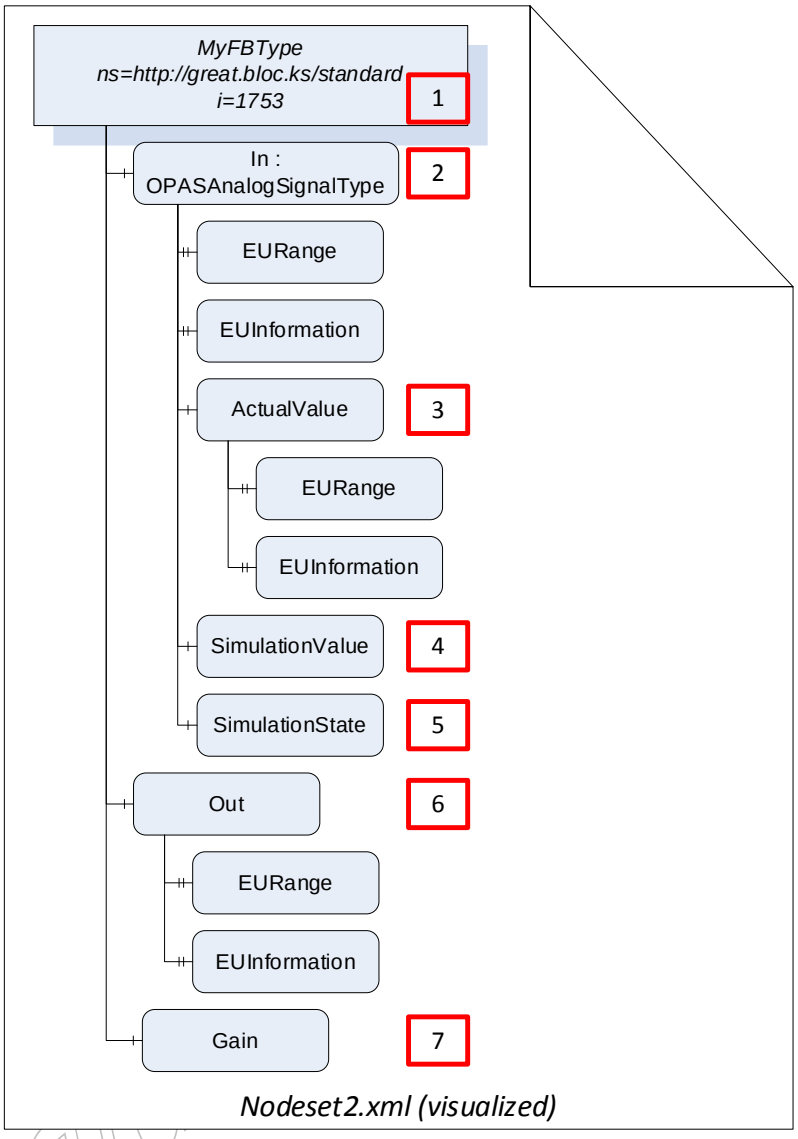
F Example of Linking Between IEC 61131-10 XML and OPC UA Nodeset2 XML (Informative)

This appendix provides an example of how to use the IEC 61131-10 XML AddData elements specified in Section 4.3.

Section F.1 shows an IEC 61131-3 function block and the IEC 61131-10 XML describing it. The IEC 61131-10 XML contains the AddData elements specified in Section 4.3 to connect it to an OPC UA object type used to represent instances of the function block in the OPC UA. The AddData elements are identified with red [[#]] markers that show to which part of the OPC UA definition this AddData element links. The OPC UA definition is contained in a Nodeset2.xml file that is visualized in Figure 23 which contains red boxes with numbers corresponding to those in the XML in Section F.1.

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution



Note: The numbers in red boxes indicate links to specific AddData elements identified with red [[#]] markers in Section F.1.

Figure 23: OPC UA Object Type Definition for a Function Block

F.1 Example IEC 61131-3 Function Block and IEC 61131-10 XML Description

```
FUNCTION_BLOCK MyFBType [[1]]
VAR_INPUT
    In : OPAS_LREAL_SIGNAL_TYPE; [[3]]
END_VAR
VAR_OUTPUT
    Out : OPAS_LREAL_SIGNAL_TYPE; [[6]]
END_VAR
VAR
    EffectiveIn : LREAL; [[2]] // input value after simulation switch.
    SimValue : LREAL; [[4]] // value of UA variable
```


© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

```
    SimStatus : OPAS_SIGNAL_STATUS_TYPE; [[4]] // status of UA Variable
    SimEn : BOOL; [[5]]
    Gain : LREAL; [[7]]
END_VAR

// Implementation omitted

END_FUNCTION_BLOCK

<FunctionBlock name="MyFBType">
  <AddData>
    <Data name="http://www.opengroup.org/open-process-automation/O-PAS/61131-
10/schema">
      <UAObjectTypeLink> [[1]]
        <NodeId>
          <NamespaceURI>http://great.bloc.ks/standard</NamespaceURI>
          <Value> <Numeric>1753</Numeric></Value>
        </NodeId>
      </UAObjectTypeLink>
    </Data>
  </AddData>
  <Parameters>
    <InputVars>
      <Variable name="In" orderWithinParamSet="1">
        <Type>  <TypeName>OPAS_LREAL_SIGNAL_TYPE</TypeName> </Type>
        <AddData>
          <Data name="http://www.opengroup.org/open-process-automation/O-
PAS/61131-10/schema">
            <UAObjectVariableLink permittedAccessLevel="21"
permittedAccessLevelScope="VariableOnly"> [[3]]
              <BroswePathSegment>
                <NamespaceURI>http://great.bloc.ks/standard</NamespaceURI>
                <BrowsePathSegmentName>In</BrowsePathSegmentName>
              </BroswePathSegment>
              <BroswePathSegment>
                <NamespaceURI>http://opcfoundation.org/UA/PADIM/</NamespaceURI>
                <BrowsePathSegmentName>ActualValue</BrowsePathSegmentName>
              </BroswePathSegment>
            </UAObjectVariableLink>
          </Data>
        </AddData>
      </Variable>
    </InputVars>
    <OutputVars>
      <Variable name="Out" orderWithinParamSet="1">
        <Type>  <TypeName>OPAS_LREAL_SIGNAL_TYPE</TypeName> </Type>
          <AddData>
            <Data name="http://www.opengroup.org/open-process-automation/O-
PAS/61131-10/schema">
            <UAObjectVariableLink permittedAccessLevel="119"
permittedAccessLevelScope="VariableAndProperties"> [[6]]
              <BroswePathSegment>
                <NamespaceURI>http://great.bloc.ks/standard</NamespaceURI>
                <BrowsePathSegmentName>Out</BrowsePathSegmentName>
              </BroswePathSegment>
            </UAObjectVariableLink>
          </Data>
        </AddData>
      </Variable>
    </OutputVars>
  </Parameters>
  <Vars>
    <Variable name="EffectiveIn">
```


© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

```
      <Type> <TypeName>LREAL</TypeName> </Type>
      <AddData>
        <Data name="http://www.opengroup.org/open-process-automation/O-
PAS/61131-10/schema">
          <UAObjectVariableLink permittedAccessLevel="3"
permittedAccessLevelScope="VariableAndProperties"> [[2]]
            <BroswePathSegment>
              <NamespaceURI>http://great.bloc.ks/standard</NamespaceURI>
              <BrowsePathSegmentName>In</BrowsePathSegmentName>
            </BroswePathSegment>
          </UAObjectVariableLink>
        </Data>
      </AddData>
    </Variable>
    <Variable name="SimValue">
      <Type> <TypeName>LREAL</TypeName> </Type>
      <AddData>
        <Data name="http://www.opengroup.org/open-process-automation/O-
PAS/61131-10/schema">
          <UAObjectVariableLink permittedAccessLevel="55"
permittedAccessLevelScope="VariableOnly"> [[4]]
            <BroswePathSegment>
              <NamespaceURI>http://great.bloc.ks/standard</NamespaceURI>
              <BrowsePathSegmentName>In</BrowsePathSegmentName>
            </BroswePathSegment>
            <BroswePathSegment>
              <NamespaceURI>http://opcfoundation.org/UA/PADIM/</NamespaceURI>
              <BrowsePathSegmentName>SimulationValue</BrowsePathSegmentName>
            </BroswePathSegment>
          </UAObjectVariableLink>
        </Data>
      </AddData>
    </Variable>
    <Variable name="SimStatus">
      <Type> <TypeName> OPAS_SIGNAL_STATUS_TYPE</TypeName> </Type>
      <AddData>
        <Data name="http://www.opengroup.org/open-process-automation/O-
PAS/61131-10/schema">
          <UAObjectVariableAttributeLink> [[4]]
            <BroswePathSegment>
              <NamespaceURI>http://great.bloc.ks/standard</NamespaceURI>
              <BrowsePathSegmentName>In</BrowsePathSegmentName>
            </BroswePathSegment>
            <BroswePathSegment>
              <NamespaceURI>http://opcfoundation.org/UA/PADIM/</NamespaceURI>
              <BrowsePathSegmentName>SimulationValue</BrowsePathSegmentName>
            </BroswePathSegment>
          </UAObjectVariableLink>
        </Data>
      </AddData>
    </Variable>
    <Variable name="SimEn">
      <Type> <TypeName>BOOL</TypeName> </Type>
      <AddData>
        <Data name="http://www.opengroup.org/open-process-automation/O-
PAS/61131-10/schema">
          <UAObjectVariableLink permittedAccessLevel="23"
permittedAccessLevelScope="VariableOnly"> [[5]]
            <BroswePathSegment>
              <NamespaceURI>http://great.bloc.ks/standard</NamespaceURI>
              <BrowsePathSegmentName>In</BrowsePathSegmentName>
            </BroswePathSegment>
            <BroswePathSegment>
```


© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

```
                <NamespaceURI>http://opcfoundation.org/UA/PADIM/</NamespaceURI>
                <BrowsePathSegmentName>SimulationState</BrowsePathSegmentName>
              </BroswePathSegment>
            </UAObjectVariableLink>
          </Data>
        </AddData>
      </Variable>
      <Variable name="Gain">
        <Type> <TypeName>LREAL</TypeName> </Type>
          <AddData>
            <Data name="http://www.opengroup.org/open-process-automation/O-
PAS/61131-10/schema">
              <UAObjectVariableLink permittedAccessLevel=”23”
permittedAccessLevelScope=”VariableOnly”> [[7]]
                <BroswePathSegment>
                  <NamespaceURI>http://great.bloc.ks/standard</NamespaceURI>
                  <BrowsePathSegmentName>Gain</BrowsePathSegmentName>
                </BroswePathSegment>
              </UAObjectVariableLink>
            </Data>
          </AddData>
        </Variable>
      </Vars>
      <MainBody>
          ...
      </MainBody>
</FunctionBlock>
```

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

G Abbreviations

For the OPAF Glossary and Abbreviations, see the definitions at: www.opengroup.org/open-process-automation-forum/glossary.

G.1 Abbreviations Specific to Part 6.6

The following abbreviations are specific to O-PAS Part 6.6 (and therefore do not appear in the OPAF Glossary referenced above).

Abbreviation	Definition
POU	Program Organization Unit

Evaluation Copy

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution

Index

.opas file 19
.opas package 19
61131-Application 23, 24
61131-Engine 23, 24
61131-FunctionBlockGroup 23
61131-XMLFileRelationship 23
AddData 19, 63, 69
AddDataInfo 65
AddressSpace 29
AML 15, 23
Class 16
configuration 5
conformance 1
conformance units 35, 46
control application 7
CTL-601 25
CTL-602 ..... 47, 68
DataTypeDecl 11
DCN 1
download/upload format 26
function block 37
FunctionBlock 14
Helper functions 58
IEC 61131-10 XML 69
IEC 61131-3 NAMESPACE 46
import/export 1
InstanceHierarchy 24
Instances 7
Layer-F applications 5
LConC 37
namespace 31
namespace metadata 30
namespaces 67
OBC-201 25
OCF 17
OCFB 30, 37
OCFB_ERROR 39
OCFB_IN 39
OCFB_IN_<Type> 43
OCFB_OUT 41
OCFB_OUT_<Type> 44
OCFB_Trigger_IN 41
OCFB_Trigger_OUT 42
OFB-002 25
O-PAS IEC 61131 application 5
O-PAS object model 6
OPAS_ACCESS_LEVEL_TYPE 56
OPAS_BOOL_SIGNAL_TYPE 48
OPAS_EU_INFO_TYPE 53
OPAS_LINT_SIGNAL_TYPE 48
OPAS_LREAL_SIGNAL_TYPE 50
OPAS_RANGE_LREAL_TYPE 52
OPAS_RANGE_REAL_TYPE 52
OPAS_REAL_SIGNAL_TYPE 50
OPAS_SIGNAL_INFO_TYPE 54
OPAS_SIGNAL_INFOBITS_TYPE 55
OPAS_SIGNAL_LIMITBITS_TYPE 55
OPAS_SIGNAL_METADATA_TYPE 51
OPAS_SIGNAL_SEVERITY_TYPE 53
OPAS_SIGNAL_STATUS_SUBCODE_TYPE 54
OPAS_SIGNAL_STATUS_TYPE 51
OPAS_ULINT_SIGNAL_TYPE 49
OPAS61131ExecutionEngineType 28
OPAS61131FunctionBlockGroupType29
OPC UA nodeset 22
OPC UA object types 27
Open Packaging Conventions 19
Parameters 12
PLCopen XML 6, 19
POU 7
PouDecl 11
ProgramInstance 10
Project 6
Resource 9
SEC-F-001 25
struct variable 34
SUC 14, 23
Types 7
UAObjectTypeLink 19
UAObjectVariableAttributeLink 21
UAObjectVariableLink 20
units of measure 45
UOM_NS_ID() 45
URIs 68
VAR_CONFIG 8
VAR_EXTERNAL 8
VAR_GLOBAL 8

© 2021 The Open Group, All Rights Reserved
Evaluation Copy. Not for redistribution